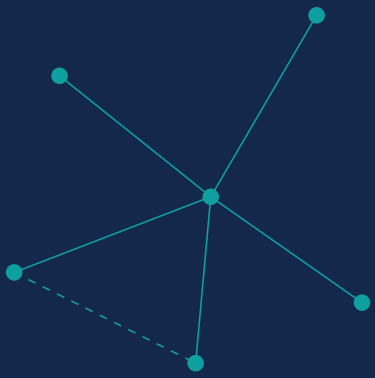


NETZWERK VERSTEHEN

Band 3

TCP UND UDP

Verbindungen, Datenübertragung, Ports und Netzwerkkommunikation verständlich erklärt

Eugen Becker

Netzwerk verstehen, Band 3
Grundlagen der Netzwerktechnik

Autor: Eugen Becker

Erste Ausgabe

Dieses Buch ist urheberrechtlich geschützt. Die Verwendung der Inhalte außerhalb der gesetzlichen Grenzen bedarf der Zustimmung des Autors.

Die Angaben in diesem Buch wurden sorgfältig geprüft. Netzwerkgeräte, Betriebssysteme und Normen entwickeln sich weiter. Prüfe bei Konfigurationsfragen immer die Dokumentation deines Herstellers oder Betriebssystems.

Inhaltsverzeichnis

Vorwort **1**

1 Die Transportschicht verstehen **2**

1.1 Was ist die Transportschicht? 2
1.2 Position im OSI- und TCP/IP-Modell 2
1.3 Aufgaben der Transportschicht 2
1.4 Kommunikation zwischen Anwendungen 3
1.5 Ende-zu-Ende-Kommunikation 3
1.6 Netzwerkebene und Transportebene 4
1.7 Zusammenhang zwischen IP, TCP und UDP 4
1.8 Segmente und Datagramme 4
1.9 Datenfluss vom Client zum Server 4

2 Ports und Socket-Kommunikation **6**

2.1 Was ist ein Port? 6
2.2 Warum werden Ports benötigt? 6
2.3 Portnummern von 0 bis 65535 6
2.4 Well-Known Ports 7
2.5 Registered Ports 7
2.6 Dynamic und Private Ports 7
2.7 Quellport und Zielport 8
2.8 Kombination aus IP-Adresse und Port 8
2.9 Socket-Grundlagen 8
2.10 TCP-Sockets und UDP-Sockets 8
2.11 Verbindung zwischen Anwendungen 8
2.12 Bekannte Dienste und ihre Ports 9

3 TCP: Transmission Control Protocol **14**

3.1 Was ist TCP? 14
3.2 Eigenschaften von TCP 14
3.3 Verbindungsorientierte Kommunikation 14
3.4 Zuverlässige Datenübertragung 15

3.5 Geordnete Datenübertragung . . . 15
3.6 Fehlererkennung . . . 15
3.7 Bestätigungen . . . 15
3.8 Wiederholungsübertragungen . . . 15
3.9 Datenstromorientierung . . . 16
3.10 Vorteile und Nachteile . . . 16
3.11 TCP-Kommunikation im Überblick . . . 16
3.12 Unterschied zwischen TCP und UDP . . . 16

4 Der TCP-Header . . . 19
4.1 Aufbau des Headers . . . 19
4.2 Source Port und Destination Port . . . 20
4.3 Sequence Number . . . 20
4.4 Acknowledgment Number . . . 20
4.5 Data Offset . . . 20
4.6 Reserved Bits . . . 21
4.7 Flags . . . 21
4.8 Window Size . . . 21
4.9 Checksum . . . 21
4.10 Urgent Pointer . . . 21
4.11 Options . . . 22
4.12 Padding . . . 22
4.13 Die Steuerflags . . . 22
4.14 Beispiel eines SYN-Pakets . . . 24
4.15 Beispiel eines ACK-Pakets . . . 24

5 TCP Three-Way Handshake . . . 26
5.1 Warum der Handshake benötigt wird . . . 26
5.2 Der Ablauf in drei Schritten . . . 26
5.3 Initial Sequence Numbers . . . 27
5.4 Synchronisierung der Sequenznummern . . . 28
5.5 Bestätigung des Verbindungsaufbaus . . . 28
5.6 Welche Informationen ausgetauscht werden . . . 28
5.7 TCP-Optionen im Handshake . . . 28
5.8 Beispiel 1: Erfolgreicher Verbindungsaufbau . . . 29
5.9 Beispiel 2: Keine Antwort auf das SYN . . . 30
5.10 Beispiel 3: Port nicht geöffnet . . . 30
5.11 Beispiel 4: Verbindungsaufbau mit Daten im Handshake . . . 30

6 TCP-Datenübertragung . . . 32
6.1 Sequenznummern im Datenstrom . . . 32
6.2 Datenübertragung in Segmenten . . . 32
6.3 Reihenfolge der Daten . . . 32

6.4 Empfangsbestätigungen . . . 33
6.5 Cumulative Acknowledgments . . . 33
6.6 Duplicate ACKs . . . 33
6.7 Out-of-Order-Segmente . . . 33
6.8 Retransmission . . . 34
6.9 Timeout . . . 34
6.10 Fast Retransmit . . . 34
6.11 Selective Acknowledgment (SACK) . . . 34
6.12 TCP-Fehlerbehandlung im Überblick . . . 35

7 Flow Control und Sliding Window . . . 37
7.1 Was ist Flow Control? . . . 37
7.2 Warum Flusskontrolle benötigt wird . . . 37
7.3 Receive Window . . . 37
7.4 Advertised Window . . . 38
7.5 Sliding Window . . . 38
7.6 Window Size . . . 38
7.7 Zero Window . . . 39
7.8 Window Update . . . 39
7.9 Buffer . . . 39
7.10 Zusammenspiel von Sender und Empfänger . . . 39

8 TCP Congestion Control . . . 41
8.1 Flow Control und Congestion Control im Vergleich . . . 41
8.2 Netzwerküberlastung . . . 41
8.3 Congestion Window . . . 42
8.4 Slow Start . . . 42
8.5 Congestion Avoidance . . . 42
8.6 Packet Loss und Reaktion . . . 42
8.7 RTT und RTO . . . 43
8.8 AIMD-Grundprinzip . . . 43
8.9 Historische und moderne Verfahren . . . 44

9 TCP-Verbindungsabbau und Zustände . . . 46
9.1 Wie wird eine TCP-Verbindung beendet? . . . 46
9.2 FIN und ACK . . . 46
9.3 Four-Way Termination . . . 46
9.4 Simultaneous Close . . . 47
9.5 RST . . . 47
9.6 Zustände einer TCP-Verbindung . . . 48
9.7 TIME_WAIT . . . 48
9.8 CLOSE_WAIT . . . 49
9.9 Weitere Zustände und Übergänge . . . 49

10 UDP: User Datagram Protocol 51
10.1 Was ist UDP? 51
10.2 Eigenschaften von UDP 51
10.3 Verbindungsloses Protokoll 52
10.4 Datagramme 52
10.5 Keine Empfangsbestätigung durch UDP selbst 52
10.6 Keine eingebaute Wiederholungsübertragung 52
10.7 Keine eingebaute Flusskontrolle 53
10.8 UDP-Checksum 53
10.9 Vorteile und Nachteile 53
10.10 Wann wird UDP verwendet? 53

11 Der UDP-Header und die UDP-Datenübertragung 55
11.1 Der Aufbau eines UDP-Datagramms 55
11.2 Source Port 55
11.3 Destination Port 56
11.4 Length 56
11.5 Checksum 56
11.6 UDP über IPv4 56
11.7 UDP über IPv6 57
11.8 Checksum-Besonderheiten 57
11.9 Fragmentierung und MTU 57
11.10 Umgang mit Paketverlust 57
11.11 Datagrammübertragung 58

12 TCP und UDP im direkten Vergleich 60
12.1 Verbindungsorientiert versus verbindungslos 60
12.2 Zuverlässigkeit und Reihenfolge 60
12.3 Geschwindigkeit und Latenz 60
12.4 Headergröße und Overhead 61
12.5 Fehlerbehandlung 61
12.6 Flow Control und Congestion Control 61
12.7 Typische Einsatzgebiete 61
12.8 Warum nicht immer dasselbe Protokoll? 61
12.9 Entscheidungshilfe 62

13 Moderne Transportprotokolle und QUIC 65
13.1 Was ist QUIC? 65
13.2 Warum wurde QUIC entwickelt? 65
13.3 QUIC und UDP 66
13.4 HTTP/3 66
13.5 TLS-Integration 66
13.6 Verbindungsaufbau 67

13.7 Multiplexing . . . 67
13.8 Head-of-Line Blocking . . . 67
13.9 Connection Migration . . . 68
13.10 TCP versus QUIC . . . 68
13.11 Einsatz im modernen Internet . . . 69

14 Praktische Netzwerkdiagnose mit Wireshark . . . 70
14.1 Wireshark installieren . . . 70
14.2 Netzwerkadapter auswählen . . . 70
14.3 Paketaufzeichnung starten . . . 71
14.4 TCP-Verkehr erkennen . . . 71
14.5 UDP-Verkehr erkennen . . . 71
14.6 TCP Three-Way Handshake untersuchen . . . 72
14.7 TCP-Verbindungsabbau beobachten . . . 72
14.8 Ports analysieren . . . 72
14.9 TCP-Retransmissions erkennen . . . 72
14.10 Duplicate ACKs erkennen . . . 72
14.11 DNS-Kommunikation untersuchen . . . 73
14.12 Filter anwenden . . . 73

15 Praktische Laborübungen . . . 75
15.1 Vorbereitung . . . 75
15.2 Windows . . . 75
15.3 Linux . . . 77

16 Zusammenfassung und Prüfungsvorbereitung . . . 82
16.1 Wichtigste TCP-Begriffe . . . 82
16.2 Wichtigste UDP-Begriffe . . . 82
16.3 TCP-Header und UDP-Header . . . 82
16.4 TCP-Handshake . . . 83
16.5 Abbau und Zustände . . . 83
16.6 QUIC . . . 83
16.7 Wichtigste Formeln und Werte . . . 83
16.8 Multiple-Choice-Fragen . . . 84
16.9 Offene Fragen . . . 84
16.10 Praxisaufgaben . . . 85
16.11 Vollständige Lösungen . . . 85

Anhang . . . 87

Vorwort

Du hast die ersten beiden Bände der Reihe hinter dir. Du weißt, wie Geräte im lokalen Netz Frames austauschen, wie IP-Adressen aufgebaut sind und wie Router Pakete weiterleiten. Dieser Band setzt genau dort an, wo IP aufhört. IP bringt ein Paket zuverlässig in ein anderes Netz, aber es garantiert nicht, dass die Nachricht vollständig, in der richtigen Reihenfolge und ohne Verluste ankommt. Diese Aufgabe übernimmt die Transportschicht, und ihre beiden wichtigsten Protokolle heißen TCP und UDP.

Wenn du eine Webseite lädst, eine Datei übertragst oder ein Video streamst, laufen diese Protokolle im Hintergrund. Sie entscheiden, ob eine Verbindung aufgebaut wird, wie verlorene Daten nachgeliefert werden und wie viele Daten gleichzeitig unterwegs sein dürfen. Wer sie versteht, kann Verbindungsprobleme gezielt eingrenzen, statt nur zu raten.

Dieses Buch ist so aufgebaut, dass du es auch ohne die Vorbände verstehen kannst. Wo ein Grundbegriff aus Band 1 oder Band 2 wichtig ist, erinnern wir kurz daran. Die Erklärungen beginnen immer mit einem Alltagsvergleich, danach folgt die technische Funktion. Viele Abläufe zeigen wir als Grafik, denn ein Verbindungsaufbau ist schwer zu beschreiben, aber leicht zu zeichnen.

Ein Tipp zum Lernen: Lies den Abschnitt über den Three-Way Handshake langsam und zeichne die drei Nachrichten selbst auf einen Zettel. Das Gleiche gilt für die Sequenznummern. Wer den Ablauf einmal von Hand durchgeht, vergisst ihn nicht mehr. Die Übungen am Ende der Kapitel sind dafür gedacht, dass du das Gelernte auf neue Fälle anwendest.

Viel Erfolg beim Lesen, Eugen Becker

KAPITEL 1

Die Transportschicht verstehen

Stell dir vor, du schickst eine Kiste mit zwanzig Büchern an eine Freundin. Die Post bringt die Kiste zuverlässig zur richtigen Adresse. Aber was passiert, wenn in der Kiste zwei Bücher fehlen, wenn ein Buch doppelt drin liegt oder wenn die Bücher in falscher Reihenfolge ankommen? Die Post kümmert sich nur um den Transport der Kiste. Ob der Inhalt vollständig ist, bleibt deine Sache, oder du organisierst dafür eine Kontrolle. Im Netz übernimmt die Transportschicht genau diese Kontrolle.

1.1 Was ist die Transportschicht?

Die Transportschicht sorgt dafür, dass Daten zwischen zwei Anwendungen auf verschiedenen Geräten ankommen. Sie sitzt oberhalb von IP und unterhalb der Anwendung, also etwa des Browsers oder eines Mailprogramms. Ihre Aufgabe ist nicht, das Paket durch das Netz zu lotsen, das erledigt IP. Sie sorgt dafür, dass die Daten am Ziel sinnvoll zusammengesetzt werden und dass die richtige Anwendung sie erhält.

1.2 Position im OSI- und TCP/IP-Modell

Im OSI-Modell belegt die Transportschicht die Schicht 4, zwischen der Vermittlungsschicht und den Sitzungs-, Darstellungs- und Anwendungsschichten. Im TCP/IP-Modell gehört sie zur Transportschicht, die direkt auf der Internetschicht mit IP aufsetzt. Die folgende Abbildung zeigt beide Modelle nebeneinander, die Transportschicht ist hervorgehoben.

1.3 Aufgaben der Transportschicht

Die Transportschicht erfüllt vier Aufgaben. Die erste ist die Zuordnung zu einer Anwendung. Auf einem Gerät laufen viele Programme gleichzeitig, und jedes eingehende Paket muss an das richtige Programm gehen. Dafür dienen Portnummern, die wir im nächsten Kapitel ausführlich behandeln. Die zweite Aufgabe ist die Zerlegung großer Datenmengen in Stücke, die im Netz gut transportiert werden können. Die dritte ist die Kontrolle,

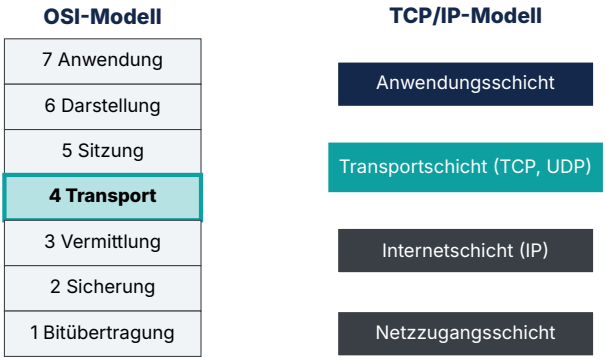


Abbildung 1.1. Die Transportschicht im OSI-Modell (Schicht 4, hervorgehoben) und im TCP/IP-Modell. Die Schichten 5 bis 7 des OSI-Modells fasst das TCP/IP-Modell in der Anwendungsschicht zusammen. Schematische Darstellung.

ob die Daten vollständig und in der richtigen Reihenfolge angekommen sind. Die vierte betrifft die Steuerung der Übertragung, damit weder der Empfänger noch das Netz überlastet werden.

Nicht jede Transportschicht erfüllt alle vier Aufgaben im gleichen Umfang. Hier liegt der Kern des Unterschieds zwischen TCP und UDP, den wir in den nächsten Kapiteln Schritt für Schritt herausarbeiten.

1.4 Kommunikation zwischen Anwendungen

Netzwerkkommunikation findet nicht zwischen Geräten statt, sondern zwischen Anwendungen. Dein Browser spricht nicht mit dem Rechner des Servers, sondern mit einem bestimmten Dienst auf diesem Rechner, zum Beispiel dem Webserver. IP-Adressen beschreiben die Geräte, Portnummern beschreiben die Anwendungen auf dem Gerät. Die Kombination aus beidem ergibt eindeutig, wohin eine Nachricht gehört.

1.5 Ende-zu-Ende-Kommunikation

Die Transportschicht arbeitet Ende zu Ende. Das bedeutet, dass nur die beiden Endpunkte der Verbindung ihre Zustände verwalten. Die Router auf dem Weg kennen die Verbindung nicht. Sie leiten einzelne Pakete weiter, ohne zu wissen, zu welcher Unterhaltung sie gehören. Gerade deshalb können Netze so groß und so flexibel sein. Die

Zuverlässigkeit entsteht an den Enden, nicht in der Mitte des Netzes.

1.6 Netzwerkebene und Transportebene

Der Unterschied zwischen beiden Ebenen lässt sich mit dem Postbeispiel gut erklären. Die Netzwerkebene, also IP, kennt die Adresse des Hauses. Die Transportebene kennt die Wohnung im Haus und prüft, ob alle Briefe angekommen sind. IP sagt, wohin das Paket gehen soll. TCP oder UDP sagen, an wen im Haus es geht und wie sicher die Zustellung sein muss.

> **Merke**
>
> IP liefert Pakete an ein Gerät. TCP oder UDP liefern Daten an eine bestimmte Anwendung auf diesem Gerät und entscheiden, wie zuverlässig die Übertragung abläuft.

1.7 Zusammenhang zwischen IP, TCP und UDP

Der Aufbau eines Pakets zeigt den Zusammenhang. Ganz außen steht der IP-Header mit den Adressen. Darin verweist das Feld Protocol auf das nächste Protokoll: Der Wert 6 bedeutet TCP, der Wert 17 bedeutet UDP. Im Inneren folgt der Header von TCP oder UDP und danach die eigentlichen Daten der Anwendung.

1.8 Segmente und Datagramme

Die Dateneinheit der Transportschicht heißt bei TCP Segment und bei UDP Datagramm. Ein TCP-Segment trägt eine Sequenznummer, eine Bestätigungsnummer und Steuerflags. Ein UDP-Datagramm ist dagegen schlichter aufgebaut und trägt nur Portnummern, eine Länge und eine Prüfsumme. Beide Einheiten landen als Nutzdaten im IP-Paket.

1.9 Datenfluss vom Client zum Server

Die folgende Abbildung zeigt den Weg einer Anfrage vom Client zum Server über alle Schichten. Die Anwendung erzeugt die Daten, die Transportschicht verpackt sie in ein Segment mit Portnummern, IP setzt die Adressen und die Netzzugangsschicht sendet den Frame.

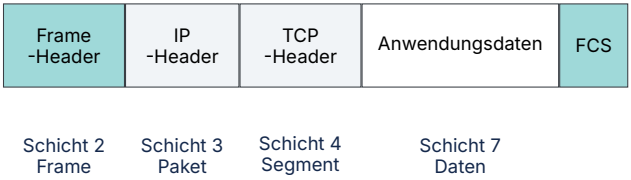


Abbildung 1.2. Aufbau einer Nachricht vom Client zum Server. Die Transportschicht fügt den TCP-Header hinzu, IP den IP-Header und die Netzzugangsschicht den Frame-Header und die Prüfsumme. Die Größen sind vereinfacht, schematische Darstellung.

Kontrollfragen

1. Welche Aufgabe hat IP, und welche Aufgabe hat die Transportschicht?
2. Auf welcher Schicht des OSI-Modells liegt die Transportschicht?
3. Welche Protokollnummer steht im IP-Header, wenn ein TCP-Segment transportiert wird?
4. Warum sagt man, dass die Transportschicht Ende zu Ende arbeitet?

Musterlösungen: 1. IP bringt Pakete zwischen Netzen zu einem Gerät. Die Transportschicht ordnet die Daten einer Anwendung zu und sorgt je nach Protokoll für Zuverlässigkeit und Steuerung. 2. Schicht 4. 3. Protokollnummer 6. 4. Nur die beiden Endpunkte verwalten den Zustand der Verbindung, die Router auf dem Weg kennen sie nicht.

KAPITEL 2

Ports und Socket-Kommunikation

Auf deinem Laptop laufen gleichzeitig ein Browser, ein Musikprogramm, ein Mailprogramm und vielleicht ein Chat. Kommt eine Nachricht aus dem Netz an, muss das System entscheiden, welches Programm sie erhält. Die IP-Adresse führt die Nachricht nur zum Gerät. Für die Zuordnung zur Anwendung braucht man eine zweite Angabe, und das ist der Port. Dieses Kapitel erklärt, was Ports sind, wie sie nummeriert werden und wie aus einer IP-Adresse und einer Portnummer eine eindeutige Verbindung entsteht.

2.1 Was ist ein Port?

Ein Port ist eine Nummer, die eine bestimmte Anwendung oder einen Dienst auf einem Gerät kennzeichnet. Man kann sich das wie eine Hausnummer innerhalb eines Gebäudes vorstellen. Die Adresse des Gebäudes entspricht der IP-Adresse, die Wohnungsnummer entspricht dem Port. Ein Brief an das Gebäude findet nur dann den richtigen Empfänger, wenn die Wohnungsnummer stimmt.

2.2 Warum werden Ports benötigt?

Ohne Ports könnte ein Gerät immer nur einen Dienst gleichzeitig bedienen. Sobald zwei Programme eine Nachricht erwarten, wüsste das System nicht, wem sie gehört. Ports lösen dieses Problem. Jeder Dienst, der auf Anfragen wartet, belegt einen Port. Jede ausgehende Verbindung bekommt ebenfalls einen Port, damit die Antworten zurückfinden.

2.3 Portnummern von 0 bis 65535

Eine Portnummer ist eine 16-Bit-Zahl. Es gibt also 65536 mögliche Werte, von 0 bis 65535. Die Internet Assigned Numbers Authority, kurz IANA, teilt diesen Bereich in drei Gruppen ein. Die Grenzen zwischen den Gruppen sind wichtig, weil sie unterschiedliche Zwecke haben.

2.4 Well-Known Ports

Die Well-Known Ports reichen von 0 bis 1023. Sie sind für allgemein bekannte Dienste reserviert, etwa Webserver auf Port 80 oder E-Mail auf Port 25. Auf Serversystemen laufen diese Dienste meist unter dem Port, den man ihnen zugeordnet hat, damit Clients sie leicht finden. Nur privilegierte Prozesse dürfen unter diesen Nummern lauschen.

2.5 Registered Ports

Die Registered Ports reichen von 1024 bis 49151. Hersteller und Entwickler können sie für ihre Anwendungen registrieren lassen. Viele Programme, etwa Datenbanken oder Spiele, verwenden Ports aus diesem Bereich.

2.6 Dynamic und Private Ports

Die Dynamic oder Private Ports reichen von 49152 bis 65535. Sie werden nicht fest zugeordnet. Wenn dein Browser eine Verbindung zu einer Webseite öffnet, wählt das System für die eigene Seite der Verbindung einen freien Port aus diesem Bereich. Diese Ports heißen deshalb auch ephemere Ports, also flüchtige Ports, denn sie verschwinden, sobald die Verbindung endet.

Die folgende Abbildung zeigt die drei Bereiche auf einer Zahlenskala.

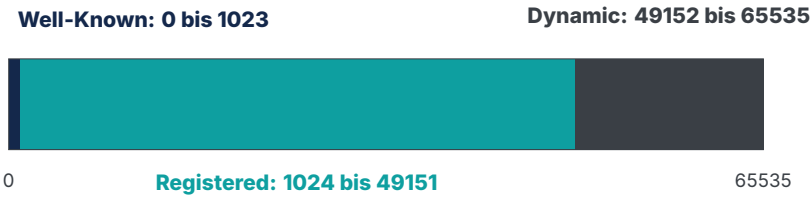


Abbildung 2.1. Die drei Bereiche der Portnummern im Maßstab. Die Well-Known Ports nehmen nur einen kleinen Teil des Bereichs ein. Der Großteil der Nummern liegt im Registered- und im Dynamic-Bereich.

2.7 Quellport und Zielport

Jedes Paket der Transportschicht trägt zwei Portnummern. Der Zielport gibt an, welche Anwendung auf dem Empfänger die Daten erhalten soll. Der Quellport gibt an, welche Anwendung auf dem Absender die Antwort erwartet. Bei einer Anfrage eines Browsers an einen Webserver ist der Zielport meist 443, und der Quellport ist ein dynamischer Port aus dem hohen Bereich.

2.8 Kombination aus IP-Adresse und Port

Eine Verbindung ist eindeutig durch vier Angaben beschrieben: die IP-Adresse und den Port des Clients sowie die IP-Adresse und den Port des Servers. Zwei Verbindungen zum selben Webserver unterscheiden sich deshalb im Quellport, auch wenn beide Clients dieselbe IP-Adresse haben. Diese Vierergruppe nennt man auch Socket-Paar.

2.9 Socket-Grundlagen

Ein Socket ist die Schnittstelle, über die eine Anwendung mit dem Netzwerk spricht. Das Betriebssystem stellt sie bereit. Ein Serverprogramm öffnet einen Socket und wartet an einem bestimmten Port auf Verbindungen. Ein Clientprogramm öffnet einen Socket und verbindet sich mit der Adresse und dem Port des Servers. Der Socket ist also die technische Stelle, an der aus einer Portnummer eine laufende Kommunikation wird.

2.10 TCP-Sockets und UDP-Sockets

Es gibt zwei Arten von Sockets. Ein TCP-Socket baut eine verbindungsorientierte Verbindung auf und hält sie während der Übertragung offen. Ein UDP-Socket sendet einzelne Datagramme, ohne vorher eine Verbindung aufzubauen. Dieselbe Portnummer kann für TCP und UDP gleichzeitig vergeben sein, denn beide Protokolle haben getrennte Portnummernräume. DNS nutzt zum Beispiel sowohl UDP als auch TCP auf Port 53.

2.11 Verbindung zwischen Anwendungen

Die folgende Abbildung zeigt einen Client, auf dem mehrere Anwendungen gleichzeitig Verbindungen aufgebaut haben. Jede Verbindung hat einen eigenen Quellport, sodass die Antworten jeweils zur richtigen Anwendung zurückfinden.

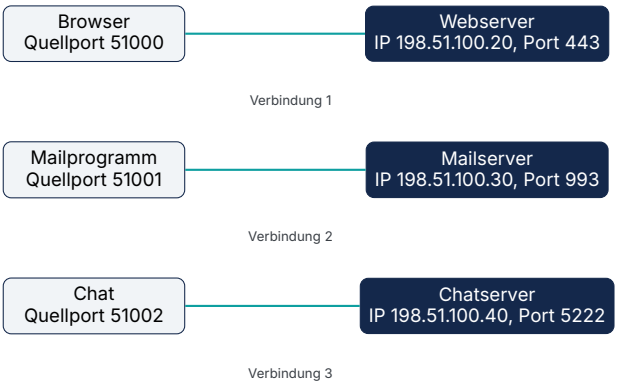


Abbildung 2.2. Ein Client mit drei gleichzeitigen Verbindungen. Jede Anwendung hat einen eigenen Quellport. Die Zielports kennzeichnen die Dienste auf den Servern. Die IP-Adressen sind Beispiele aus dem Dokumentationsbereich. Schematische Darstellung.

2.12 Bekannte Dienste und ihre Ports

Viele Ports sind über Jahrzehnte mit bestimmten Diensten verbunden. Die folgende Tabelle zeigt die wichtigsten, die du im Alltag und in der Ausbildung immer wieder brauchst. Die Erklärung der einzelnen Dienste folgt jeweils darunter.

Port 20 und 21: FTP

FTP, das File Transfer Protocol, trennt Steuerung und Datenübertragung. Über Port 21 laufen die Befehle, etwa Anmelden oder Dateiliste anfordern. Die eigentlichen Dateien fließen über Port 20. Im aktiven Modus baut der Server die Datenverbindung selbst auf, im passiven Modus baut der Client sie auf. FTP überträgt Passwörter im Klartext, deshalb gilt es heute als unsicher und wird oft durch SFTP oder FTPS ersetzt.

Port 22: SSH

SSH, Secure Shell, öffnet eine verschlüsselte Verbindung zu einem entfernten Rechner, etwa zur Fernwartung eines Linux-Servers. Der Verbindungsaufbau wird durch TCP abgesichert, die Inhalte durch Verschlüsselung geschützt. SSH ist eines der wichtigsten Werkzeuge in der Administration.

Port 23: Telnet

Telnet bietet eine Fernanmeldung ohne Verschlüsselung. Passwörter und Befehle laufen im Klartext über das Netz. Telnet ist daher nur noch in alten Systemen anzutreffen. Wer Fernzugriff braucht, sollte SSH verwenden.

Port 25: SMTP

SMTP, das Simple Mail Transfer Protocol, ist das Protokoll für den Versand von E-Mails zwischen Servern. Ein Mailprogramm übergibt seine Nachricht an den Ausgangsserver, der sie über Port 25 an den Empfängerserver weitergibt. Für den Abruf beim Empfänger dienen andere Protokolle wie POP3 und IMAP.

Port 53: DNS

DNS, das Domain Name System, übersetzt Namen wie www.example.com in IP-Adressen. Kurze Anfragen laufen über UDP, weil der Overhead gering ist. Größere Antworten und Zonenübertragungen nutzen TCP auf demselben Port. Deshalb muss man beide Protokolle auf Port 53 erlauben, wenn der DNS-Verkehr funktionieren soll.

Port 67 und 68: DHCP

DHCP vergibt IP-Konfigurationen an Geräte. Der Server lauscht auf Port 67, der Client auf Port 68. Die Anfragen des Clients kommen oft als Broadcast an, weil das Gerät vorher noch keine Adresse besitzt. Das ist ein gutes Beispiel dafür, dass UDP in der Praxis gerade für kurze, einfache Anfragen ideal ist.

Port 80: HTTP

HTTP überträgt Webseiten im Klartext. Der Browser sendet eine Anfrage, der Webserver antwortet mit dem Inhalt. Weil der Inhalt unverschlüsselt ist, wird HTTP für sensible Daten zunehmend durch HTTPS ersetzt.

Port 110: POP3

POP3 holt E-Mails vom Postfach ab und löscht sie meist vom Server, sobald sie auf dem Gerät liegen. Es eignet sich für Anwender, die ihre Mails nur auf einem Gerät lesen. Für mehrere Geräte ist IMAP besser geeignet.

Port 123: NTP

NTP, das Network Time Protocol, synchronisiert die Uhrzeit von Geräten mit Zeitservern. Es läuft über UDP, weil eine kurze Antwort genügt und Wiederholungen die Genauigkeit eher verschlechtern würden. Genaue Zeit ist wichtig für Logdateien, Zertifikate und Kerberos.

Port 143: IMAP

IMAP lässt das Mailprogramm die Nachrichten direkt auf dem Server verwalten. Ordner, Gelesen-Status und Suche bleiben zentral. Dadurch sehen alle Geräte denselben Stand. IMAP über Port 143 ist unverschlüsselt, die verschlüsselte Variante läuft meist auf Port 993.

Port 443: HTTPS

HTTPS ist HTTP mit TLS-Verschlüsselung. Der Browser baut zuerst eine TCP-Verbindung auf, danach folgt die Verschlüsselung und erst dann die Anfrage. Port 443 ist heute der wichtigste Port im Internet, weil fast alle Webseiten darüber laufen.

Port 445: SMB

SMB, Server Message Block, ist das Protokoll für Datei- und Druckerfreigaben unter Windows. Der Zugriff auf Freigaben im Netz läuft über diesen Port. Weil SMB häufig von Schadsoftware angegriffen wird, sollte er nie aus dem Internet erreichbar sein.

Port 3389: RDP

RDP, das Remote Desktop Protocol, ermöglicht den grafischen Fernzugriff auf einen Windows-Rechner. Der Port sollte nicht ungeschützt ins Internet geöffnet werden, weil er ein beliebtes Ziel von Angriffen ist. Der Zugriff sollte über ein VPN oder einen abgesicherten Zugang erfolgen.

> **Merke**
>
> Eine IP-Adresse beschreibt das Gerät, ein Port die Anwendung darauf. Eine Verbindung ist durch Quelladresse, Quellport, Zieladresse und Zielport eindeutig bestimmt.

Kontrollfragen

1. In welchem Bereich liegt die Portnummer 8080, und zu welcher Gruppe gehört sie?
2. Warum erhält eine ausgehende Verbindung meist einen Port aus dem hohen Bereich?
3. Welcher Dienst läuft auf Port 993, und in welchem Verhältnis steht er zu Port 143?
4. Warum genügt eine IP-Adresse nicht, um zwei gleichzeitige Verbindungen zu einem Webserver zu unterscheiden?

Musterlösungen: 1. 8080 liegt im Registered-Bereich von 1024 bis 49151. Er wird oft für alternative Webdienste verwendet. 2. Der Quellport ist nur für die Dauer der Verbindung gültig und muss nicht fest vergeben sein. Das System wählt einen freien Port aus dem dynamischen Bereich. 3. Port 993 gehört zu IMAP über TLS, also der verschlüsselten Variante von IMAP auf Port 143. 4. Beide Verbindungen haben dieselbe IP-Adresse als Ziel. Erst der unterschiedliche Quellport macht sie eindeutig.

Port	Protokoll	Dienst	Transport
20, 21	TCP	FTP, Daten und Steuerung	TCP
22	TCP	SSH	TCP
23	TCP	Telnet	TCP
25	TCP	SMTP, E-Mail-Versand	TCP
53	TCP, UDP	DNS	TCP und UDP
67, 68	UDP	DHCP, Server und Client	UDP
80	TCP	HTTP, Webseiten	TCP
110	TCP	POP3, E-Mail-Abruf	TCP
123	UDP	NTP, Zeitsynchronisation	UDP
143	TCP	IMAP, E-Mail-Abruf	TCP
443	TCP	HTTPS, verschlüsselte Webseiten	TCP
445	TCP	SMB, Dateifreigaben	TCP
3389	TCP	RDP, Remotedesktop	TCP

Tabelle 2.1. Wichtige Dienste und ihre Standardports. Die Angaben beruhen auf der Zuordnung der IANA.

KAPITEL 3

TCP: Transmission Control Protocol

Wenn du einem Freund einen wichtigen Vertrag per Post schickst, verlässt du dich nicht darauf, dass er einfach irgendwann ankommt. Du verschickst ihn als Einschreiben, und der Empfänger muss unterschreiben. Fehlt eine Seite, merkst du es und schickst sie nach. TCP arbeitet nach demselben Prinzip. Es sorgt dafür, dass Daten vollständig, in der richtigen Reihenfolge und ohne Fehler ankommen, und es meldet dem Absender, wenn etwas verloren ging.

3.1 Was ist TCP?

TCP, das Transmission Control Protocol, ist das wichtigste verbindungsorientierte Transportprotokoll im Internet. Es ist in RFC 793 beschrieben, die Grundfunktionen sind seit Jahrzehnten stabil geblieben. Viele Anwendungen, die Verlässlichkeit brauchen, bauen auf TCP auf, darunter Webseiten, E-Mail und Dateiübertragungen.

3.2 Eigenschaften von TCP

Die wichtigsten Eigenschaften von TCP lassen sich in wenigen Begriffen zusammenfassen. TCP baut eine Verbindung auf, bevor Daten fließen. Es bestätigt den Empfang. Es nummeriert die Daten, damit der Empfänger sie sortieren kann. Es erkennt Verluste und fordert fehlende Teile nach. Es steuert die Menge der gleichzeitig gesendeten Daten, damit der Empfänger nicht überfordert wird. Diese Eigenschaften machen TCP zuverlässig, kosten aber Zeit und Verwaltungsaufwand.

3.3 Verbindungsorientierte Kommunikation

Bevor TCP Daten sendet, muss zwischen Client und Server eine Verbindung entstehen. Das ähnelt einem Telefonat. Man wählt, der andere nimmt ab, und beide bestätigen, dass sie bereit sind zu sprechen. Erst danach beginnt das eigentliche Gespräch. Den Aufbau dieser Verbindung erklären wir im Kapitel über den Three-Way Handshake.

3.4 Zuverlässige Datenübertragung

Zuverlässig bedeutet bei TCP nicht, dass das Netz nie Pakete verliert. Es bedeutet, dass der Verlust bemerkt und behoben wird. Der Empfänger bestätigt jedes erfolgreich empfangene Stück. Fehlt eine Bestätigung, sendet der Absender das Stück erneut. Der Anwender merkt davon meist nichts, außer einer etwas längeren Ladezeit.

3.5 Geordnete Datenübertragung

Pakete können auf unterschiedlichen Wegen durch das Netz laufen und daher in anderer Reihenfolge eintreffen. TCP nummeriert jedes Byte der Daten fortlaufend. Der Empfänger sortiert die Teile anhand dieser Nummern, bevor er sie an die Anwendung weitergibt. Die Anwendung sieht deshalb einen geordneten Datenstrom, egal wie das Netz die Pakete tatsächlich verteilt hat.

3.6 Fehlererkennung

Mit einer Prüfsumme im Header erkennt TCP Übertragungsfehler, die auf dem Weg entstanden sind. Ein Segment mit falscher Prüfsumme wird verworfen, als wäre es nie angekommen. Der Absender bemerkt den fehlenden Empfang und sendet es erneut. Die Prüfsumme schützt also den Inhalt, nicht die Verbindung im Ganzen.

3.7 Bestätigungen

Jedes Segment, das Daten transportiert, trägt eine Bestätigungsnummer. Sie gibt an, bis zu welchem Byte der Empfänger alles vollständig erhalten hat. Diese Nummer wird meist in Antworten des Empfängers mitgeschickt, sodass Bestätigung und eigene Daten oft zusammen übertragen werden. Der Absender weiß dadurch genau, welche Teile angekommen sind.

3.8 Wiederholungsübertragungen

Bleibt eine Bestätigung aus, wartet der Absender eine bestimmte Zeit und sendet das fehlende Stück erneut. Diese Wiederholung heißt Retransmission. Wie lange gewartet wird, hängt von der gemessenen Laufzeit der Verbindung ab. Das Verfahren dazu be-

handeln wir in Kapitel 6.

3.9 Datenstromorientierung

TCP behandelt die übertragenen Daten als kontinuierlichen Strom von Bytes, nicht als Folge einzelner Nachrichten. Die Anwendung schreibt Bytes in den Strom, TCP entscheidet selbst, in welche Segmente sie zerlegt werden. Der Empfänger liest den Strom, ohne die ursprünglichen Grenzen der Schreibvorgänge zu sehen. Anwendungen, die Nachrichtengrenzen brauchen, müssen diese selbst kennzeichnen.

3.10 Vorteile und Nachteile

Die Vorteile von TCP liegen auf der Hand: Verlustfreie und geordnete Übertragung, Schutz vor Überlastung des Empfängers und ein einheitliches Verhalten für viele Anwendungen. Die Nachteile sind der Aufbau der Verbindung, die Bestätigungen und die Wiederholungen. Sie verursachen zusätzliche Verzögerung und zusätzlichen Datenverkehr. Für Anwendungen, bei denen ein verspätetes Paket wertlos ist, wie bei einer Sprachübertragung, ist TCP daher oft ungeeignet.

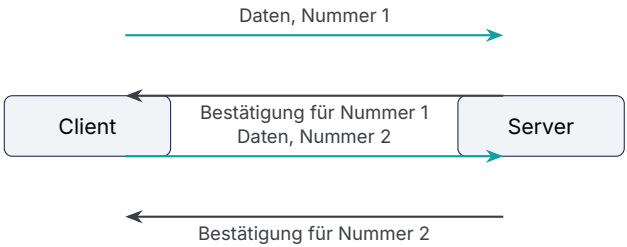
3.11 TCP-Kommunikation im Überblick

Die folgende Abbildung zeigt eine typische TCP-Kommunikation. Die Anfrage des Clients wird vom Server bestätigt, die Antwort wiederum vom Client. Jede Nachricht trägt Sequenz- und Bestätigungsnummern, die später im Kapitel 4 erklärt werden.

3.12 Unterschied zwischen TCP und UDP

TCP und UDP stehen nebeneinander, aber sie lösen unterschiedliche Probleme. Die folgende Tabelle stellt die wichtigsten Unterschiede gegenüber. UDP behandeln wir ausführlich in Kapitel 10.

> **Merke**
>
> TCP stellt sicher, dass Daten vollständig und in der richtigen Reihenfolge ankommen. Der Preis dafür sind Verbindungsaufbau, Bestätigungen und Wiederholun-



Der Absender wartet nicht auf jede Bestätigung einzeln, wenn mehrere Segmente gleichzeitig unterwegs sind.

Abbildung 3.1. Datenübertragung mit TCP über eine bestehende Verbindung. Jedes Segment wird vom Empfänger bestätigt. Schematische Darstellung, die Nummern sind vereinfacht.

gen.

Kontrollfragen

1. Welche drei Eigenschaften macht TCP zuverlässig?
2. Warum nummeriert TCP jedes Byte und nicht jedes Segment?
3. Was bedeutet es, dass TCP eine Datenstromorientierung hat?
4. Nenne eine Anwendung, bei der ein verspätetes Paket wertlos wäre, und erkläre, warum TCP dort ungeeignet ist.

Musterlösungen: 1. Verbindungsaufbau, Bestätigung des Empfangs und Wiederholung verlorener Daten. Außerdem Reihenfolge und Fehlererkennung. Zwei dieser Punkte reichen als Antwort. 2. Die Byte-Nummer ermöglicht, Daten exakt zu sortieren und Lücken auf Byte-Ebene zu erkennen. Segmente können unterschiedlich groß sein, deshalb wäre eine Nummer pro Segment ungenau. 3. Die Anwendung sieht nur einen Strom von Bytes. Nachrichtengrenzen gehen verloren und müssen von der Anwendung selbst markiert werden. 4. Sprachübertragung. Ein verspätetes Paket wäre bei einem Telefonat nicht mehr hörbar, aber TCP würde es trotzdem nachsenden und die Wiedergabe verzögern.

Merkmal	TCP	UDP
Verbindung	aufgebaut vorher	keine Verbindung
Zuverlässigkeit	Bestätigung und Wiederholung	keine eingebaute Sicherung
Reihenfolge	garantiert	nicht garantiert
Header	mindestens 20 Byte	8 Byte
Flusskontrolle	ja	nein
Einsatz	Web, Mail, Dateien	DNS, Streaming, Sprache

Tabelle 3.1. Die wichtigsten Unterschiede zwischen TCP und UDP.

KAPITEL 4

Der TCP-Header

Jedes TCP-Segment beginnt mit einem Header. Er ist wie das Anschreiben eines Briefes: Dort steht, von wem die Nachricht kommt, für wen sie bestimmt ist, welche Nummer sie trägt und welche Hinweise der Empfänger beachten muss. Der Inhalt des Briefes ist die Anwendung, der Header dagegen gehört zur Transportschicht. In diesem Kapitel gehen wir jedes wichtige Feld durch, dann die Steuerflags und schließlich zwei vollständige Beispielpakete.

4.1 Aufbau des Headers

Der TCP-Header ist mindestens 20 Byte groß, also 160 Bit. Er besteht aus festen Feldern und einem optionalen Bereich mit Erweiterungen. Die folgende Abbildung zeigt die ersten fünf Zeilen des Headers mit je 32 Bit. Jede Zeile hat vier Byte.

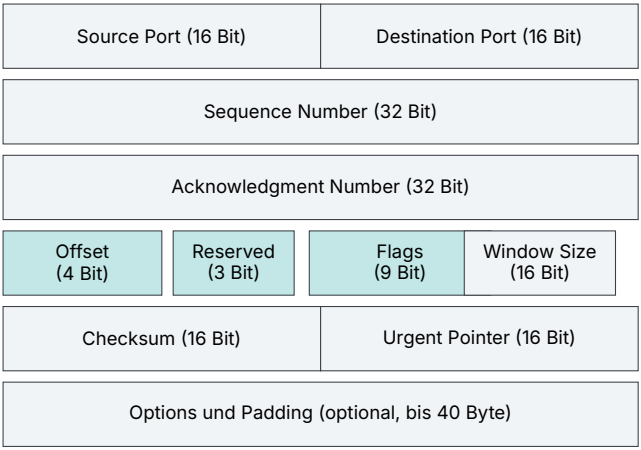


Abbildung 4.1. Die Felder des TCP-Headers in Zeilen zu je 32 Bit. Die türkisfarbenen Felder bilden die Kontrollinformationen in der dritten Zeile. Schematische Darstellung, nicht maßstabsgetreu.

Die Zeilen sind nicht nur ein Schema. Ein Empfänger liest den Header genau in dieser

Reihenfolge, und jedes Feld liegt an einer festen Stelle. Das ist der Grund, warum Analysewerkzeuge wie Wireshark Felder so zuverlässig anzeigen können.

4.2 Source Port und Destination Port

Die beiden Portfelder enthalten je 16 Bit und bezeichnen die Anwendungen auf den beiden Seiten, wie wir in Kapitel 2 gesehen haben. Der Source Port steht beim Absender und ist bei einer Anfrage meist ein dynamischer Port. Der Destination Port zeigt auf den Dienst beim Empfänger, etwa 443 für HTTPS. Bei der Antwort vertauschen sich beide Felder.

4.3 Sequence Number

Die Sequenznummer zählt die Bytes der Daten, die der Absender sendet. Sie ist 32 Bit groß und beginnt nicht bei null, sondern bei einer zufälligen Startzahl, der Initial Sequence Number. Jedes Byte der Nutzdaten erhöht die Nummer um eins. Ein Segment mit der Sequenznummer 1000 und 500 Byte Daten endet mit dem Byte 1499, das nächste Segment beginnt also bei 1500.

Eine Besonderheit gilt für Steuerflags wie SYN und FIN. Sie verbrauchen selbst eine Sequenznummer, obwohl sie keine Nutzdaten tragen.

4.4 Acknowledgment Number

Die Bestätigungsnummer gibt an, welches Byte der Absender als Nächstes erwartet. Sie gilt nur, wenn das Flag ACK gesetzt ist. Steht zum Beispiel die Bestätigungsnummer 1500, heißt das: Alle Bytes bis 1499 sind angekommen, der Empfänger wartet auf Byte 1500. Diese Angabe beschreibt also den Stand des Empfängers.

4.5 Data Offset

Der Data Offset ist 4 Bit groß und gibt die Länge des Headers in Wörtern zu je 32 Bit an. Der Wert 5 bedeutet einen Header von 20 Byte ohne Optionen. Der größtmögliche Wert 15 ergibt 60 Byte, also bis zu 40 Byte Optionen. Ohne dieses Feld könnte der Empfänger nicht erkennen, wo die Nutzdaten beginnen, weil die Optionen variabel lang sind.

4.6 Reserved Bits

Die drei Reserved-Bits sind für künftige Verwendung vorgesehen. Sie müssen auf null gesetzt werden. Ein Teil davon wird inzwischen für das Flag NS genutzt, das wir unten besprechen.

4.7 Flags

Die Flags sind einzelne Bits, die den Zustand der Verbindung steuern. Jedes Flag hat eine klare Bedeutung, und mehrere können gleichzeitig gesetzt sein. Die wichtigsten behandeln wir im nächsten Abschnitt.

4.8 Window Size

Die Window Size zeigt, wie viele Bytes der Empfänger ab der Bestätigungsnummer noch aufnehmen kann. Sie ist 16 Bit groß und erlaubt damit höchstens 65535 Byte. Über die Option Window Scaling lässt sich dieser Wert vergrößern, darauf kommen wir im Kapitel 5 zurück. Das Feld ist die Grundlage der Flusskontrolle, die Kapitel 7 ausführlich behandelt.

4.9 Checksum

Die Checksum prüft Header und Nutzdaten auf Übertragungsfehler. Sie wird wie bei UDP über einen sogenannten Pseudoheader berechnet, der Quell- und Zieladresse aus dem IP-Header enthält. Dadurch merkt man auch, wenn ein Paket mit falscher Adresse zugestellt wurde. Empfänger verwerfen Segmente mit falscher Prüfsumme.

4.10 Urgent Pointer

Der Urgent Pointer zeigt auf Daten, die vorrangig behandelt werden sollen, wenn das Flag URG gesetzt ist. Das Verfahren wird in der Praxis selten genutzt, weil viele Anwendungen es nicht unterstützen und es zu Fehlinterpretationen führen kann. Für das Verständnis genügt es, den Zweck zu kennen.

4.11 Options

Optionen erweitern den Header um zusätzliche Informationen. Sie stehen nur während des Verbindungsaufbaus oder in bestimmten Segmenten. Jede Option besteht aus einer Art, einer Länge und den Werten. Zwei Optionen sind besonders wichtig: MSS, die maximale Segmentgröße, und Window Scaling. Im Kapitel über den Handshake sehen wir sie im Detail.

4.12 Padding

Der Header muss an einer 32-Bit-Grenze enden. Ist die Summe der Optionen kein Vielfaches von vier, füllt Padding die Lücke mit Nullen auf. Die Angabe im Data Offset berücksichtigt das Padding.

4.13 Die Steuerflags

Die Flags sind die Sprache, in der TCP seinen Zustand mitteilt. Die folgende Tabelle nennt die wichtigsten Flags mit ihrer Bedeutung. Danach erklären wir, welche davon im praktischen Betrieb täglich vorkommen.

SYN

Das Flag SYN startet eine Verbindung. Es wird nur beim ersten Segment des Verbindungsaufbaus und in der Antwort des Servers gesetzt. Nach dem Handshake trägt kein weiteres Segment dieses Flag.

ACK

Das Flag ACK zeigt, dass die Bestätigungsnummer gültig ist. Fast alle Segmente nach dem Verbindungsaufbau tragen es. Ein Segment nur mit ACK und ohne Daten ist eine reine Bestätigung.

FIN

FIN signalisiert, dass eine Seite keine Daten mehr senden will. Der Abbau erfolgt in beide Richtungen getrennt. Im Kapitel 9 sehen wir den vollständigen Ablauf.

Flag	Bedeutung
SYN	Verbindungsaufbau, synchronisiert die Sequenznummern
ACK	Bestätigungsnummer ist gültig
FIN	Absender hat keine Daten mehr, beendet seine Seite
RST	Verbindung sofort abbrechen, kein sauberer Abbau
PSH	Empfänger soll die Daten sofort an die Anwendung weitergeben
URG	Urgent Pointer ist gültig
ECE	Explicit Congestion Notification, Überlastmeldung
CWR	Congestion Window Reduced, Bestätigung der Verkleinerung
NS	Nonce Sum, Schutz der ECN-Rückmeldung

Tabelle 4.1. Die TCP-Flags und ihre Bedeutung. ECE, CWR und NS gehören zur Überlastmeldung über ECN.

RST

RST bricht eine Verbindung sofort ab. Ein Gerät sendet RST zum Beispiel, wenn an dem angefragten Port kein Dienst lauscht. Eine Verbindung, die mit RST endet, ist also nicht sauber geschlossen. Die Anwendung erfährt davon durch einen Fehler.

PSH

PSH fordert den Empfänger auf, die Daten ohne Verzögerung an die Anwendung zu übergeben. Viele Systeme setzen es bei jedem Segment mit Nutzdaten, das Verhalten ist aber nur ein Hinweis.

URG

URG zeigt an, dass der Urgent Pointer gültig ist. Es ist in der Praxis selten zu sehen.

ECE, CWR und NS

ECE und CWR gehören zur expliziten Überlastmeldung. Ein Router kann ein Paket markieren, statt es zu verwerfen. Der Empfänger meldet diese Markierung über ECE an den Absender, der daraufhin mit CWR bestätigt, dass er die Senderate verringert hat. NS schützt diesen Mechanismus gegen Manipulation. Diese Flags sind im Alltag nur bei moderner Netzwerkkonfiguration relevant.

4.14 Beispiel eines SYN-Pakets

Ein Browser möchte eine Verbindung zu einem Webserver aufbauen. Das erste Paket ist ein SYN. Die folgende Tabelle zeigt die wichtigsten Felder mit Beispielwerten. Die Adressen stammen aus den Dokumentationsbereichen.

Feld	Wert
Source Port	51000
Destination Port	443
Sequence Number	1000
Acknowledgment Number	0, nicht gültig
Flags	SYN
Window Size	64240
Optionen	MSS 1460, Window Scale 7, SACK Permitted, Timestamps

Tabelle 4.2. Beispiel eines SYN-Pakets vom Client an den Server.

Die Bestätigungsnummer ist null, weil der Client noch nichts vom Server empfangen hat und daher nichts bestätigen kann. Das Flag ACK fehlt aus demselben Grund.

4.15 Beispiel eines ACK-Pakets

Ein Segment mit dem Flag ACK ist die Bestätigung, die später einen Datenstrom begleitet. Das folgende Beispiel zeigt eine reine Bestätigung nach dem Empfang von 500 Byte Daten.

Die Bestätigungsnummer 1501 belegt eine Stelle, die man leicht übersieht. Der Client

Feld	Wert
Source Port	443
Destination Port	51000
Sequence Number	5001
Acknowledgment Number	1501
Flags	ACK
Window Size	65535
Optionen	keine

Tabelle 4.3. Beispiel eines ACK-Pakets vom Server. Die Bestätigungsnummer 1501 zeigt an, dass der Server alle Bytes bis 1500 erhalten hat.

hatte 500 Byte ab Sequenznummer 1001 gesendet, also bis Byte 1500. Die Bestätigung nennt das nächste erwartete Byte, daher 1501.

> **Merke**
>
> Der TCP-Header enthält Ports für die Anwendungen, Sequenznummern für die Reihenfolge, Bestätigungsnummern für den Empfang und Flags für den Zustand der Verbindung. Die Window Size regelt, wie viel der Empfänger aufnehmen kann.

Kontrollfragen

1. Welche Länge hat ein TCP-Header ohne Optionen, und welcher Wert steht dann im Data Offset?
2. Ein Segment hat die Sequenznummer 2000 und 1200 Byte Nutzdaten. Welche Sequenznummer trägt das nächste Segment?
3. Welches Flag startet eine Verbindung, und welches Flag bricht sie sofort ab?
4. Warum ist die Bestätigungsnummer im ersten SYN-Paket null?

Musterlösungen: 1. 20 Byte, also der Wert 5 im Data Offset. 2. 2000 plus 1200 ergibt 3200, das nächste Segment beginnt bei 3200. 3. SYN startet, RST bricht sofort ab. 4. Der Client hat noch kein Byte vom Server empfangen und kann deshalb nichts bestätigen. Das Flag ACK fehlt aus demselben Grund.

KAPITEL 5

TCP Three-Way Handshake

Bevor ein Browser eine Webseite lädt, passiert etwas, das man nicht sieht: Client und Server verständigen sich darauf, dass sie miteinander reden wollen und mit welchen Startwerten. Dieser Vorgang heißt Three-Way Handshake, zu Deutsch Dreiwege-Handschlag. Er besteht aus drei Nachrichten, und er ist die Grundlage jeder TCP-Verbindung. Dieses Kapitel erklärt den Ablauf Schritt für Schritt, zeigt die Startwerte und die Optionen und durchspielt mehrere Beispiele.

5.1 Warum der Handshake benötigt wird

Stell dir ein Telefonat vor. Du rufst an, der andere hebt ab und sagt Hallo. Du antwortest, dass du ihn hörst. Erst dann beginnt das eigentliche Gespräch, weil beide sicher sind, dass die Leitung funktioniert und beide Seiten zuhören. Ohne diesen Austausch könnte der Angerufene gar nicht wissen, ob seine Antwort je ankommt.

TCP braucht den Handshake aus drei Gründen. Erstens müssen beide Seiten ihre Sequenznummern abgleichen, damit sie wissen, mit welcher Zahl die Nummerierung beginnt. Zweitens prüfen beide, dass die Verbindung in beide Richtungen funktioniert. Drittens handeln sie die Optionen aus, die für die Verbindung gelten sollen, etwa die maximale Segmentgröße.

5.2 Der Ablauf in drei Schritten

Der Client sendet zuerst ein Segment mit dem Flag SYN. Der Server antwortet mit einem Segment, das gleichzeitig SYN und ACK trägt, also SYN-ACK. Schließlich bestätigt der Client mit einem ACK. Danach gilt die Verbindung als aufgebaut, und die Anwendungsdaten können fließen.

Die folgende Abbildung zeigt den zeitlichen Ablauf zwischen Client und Server. Die Zeit läuft von oben nach unten.

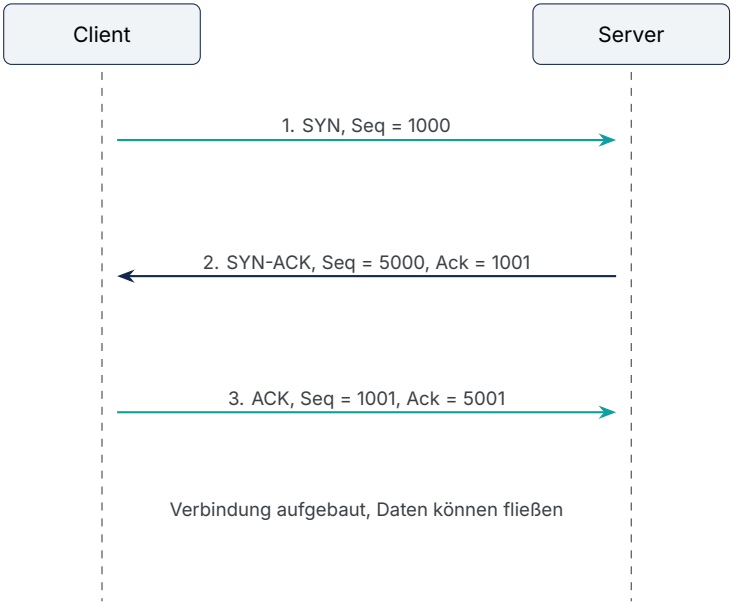


Abbildung 5.1. Zeitlicher Ablauf des Three-Way Handshakes. Der Client sendet SYN, der Server antwortet mit SYN-ACK, der Client bestätigt mit ACK. Die Werte sind Beispiele. Schematische Darstellung.

5.3 Initial Sequence Numbers

Jede Seite wählt beim Verbindungsaufbau eine Startnummer, die Initial Sequence Number, kurz ISN. Die Nummer ist nicht bei null, sondern zufällig gewählt. Dadurch lassen sich Segmente alter Verbindungen schlechter mit neuen verwechseln, und Angreifer können Verbindungen nur schwer fälschen. Heutige Systeme erzeugen die ISN nach RFC 6528 mit kryptografisch unvorhersehbaren Verfahren.

Im Beispiel wählt der Client die ISN 1000, der Server die ISN 5000. Der Client sendet im ersten Segment Seq = 1000. Der Server bestätigt mit Ack = 1001, weil das SYN selbst eine Sequenznummer verbraucht. Das Flag SYN zählt also wie ein Byte.

5.4 Synchronisierung der Sequenznummern

Im ersten Schritt teilt der Client seine ISN mit. Im zweiten Schritt teilt der Server seine ISN mit und bestätigt die des Clients, indem er die Bestätigungsnummer auf ISN plus eins setzt. Im dritten Schritt bestätigt der Client die ISN des Servers mit ACK 5001. Danach kennen beide Seiten beide Startnummern.

Dieser Austausch hat eine wichtige Eigenschaft. Jede Seite bestätigt die Nummer der anderen. So ist sichergestellt, dass beide Richtungen funktionieren, bevor Nutzdaten fließen.

5.5 Bestätigung des Verbindungsaufbaus

Warum sind drei Nachrichten nötig und nicht zwei? Der Server kann nach dem ersten SYN nicht wissen, ob seine Antwort den Client erreicht. Erst wenn der Client mit ACK antwortet, weiß der Server, dass die Verbindung in beide Richtungen steht. Mit zwei Nachrichten bliebe der Server im Ungewissen, und alte, verspätete SYN-Pakete könnten Verbindungen öffnen, die niemand mehr nutzt.

5.6 Welche Informationen ausgetauscht werden

Im Handshake gehen mehrere Angaben hin und her: die ISN beider Seiten, die Bestätigungsnummern, die Fenstergröße und die Optionen. Die Fenstergröße im SYN zeigt, wie viele Daten der Absender zu Beginn empfangen kann. Die Optionen legen fest, welche Erweiterungen beide Seiten nutzen. Alles Weitere verhandelt TCP erst während der Übertragung.

5.7 TCP-Optionen im Handshake

Optionen werden fast ausschließlich in den Handshake-Segmenten ausgehandelt. Die vier wichtigsten sind MSS, Window Scaling, SACK Permitted und Timestamps. Jede Option besteht aus einem Typ, einer Länge und dem Wert.

MSS, Maximum Segment Size

Die MSS gibt die größte Nutzdatenmenge an, die der Absender in einem Segment empfangen kann. Typisch ist 1460 Byte bei Ethernet, denn 1500 Byte MTU minus 20 Byte

IP-Header und 20 Byte TCP-Header ergeben 1460. Ohne Angabe gilt ein kleinerer Standardwert. Die MSS verhindert, dass im Netz Fragmentierung nötig wird.

Window Scaling

Das Window-Size-Feld hat nur 16 Bit und reicht für moderne Verbindungen oft nicht aus. Die Option Window Scaling multipliziert den Wert mit einer Zweierpotenz. Der Faktor wird als Shift-Count übertragen, er kann Werte von 0 bis 14 annehmen. Mit dem Faktor 7 wird das Fenster 128-mal größer. Jede Seite verwendet ihren eigenen Faktor für die Fenster, die sie anbietet.

SACK Permitted

Mit SACK Permitted zeigen beide Seiten, dass sie die selektive Bestätigung unterstützen. Mit dieser Erweiterung kann der Empfänger genau mitteilen, welche Bytesbereiche angekommen sind. Das hilft, wenn mehrere Segmente verloren gehen. Die Option ist im Kapitel 6 im Zusammenhang mit der Wiederholung wichtig.

Timestamps

Die Timestamp-Option trägt in jedem Segment einen Zeitstempel. Damit misst der Absender die Laufzeit genauer und kann Antworten auf alte Segmente erkennen. Sie ist in RFC 7323 beschrieben und wird heute von den meisten Systemen aktiviert.

5.8 Beispiel 1: Erfolgreicher Verbindungsaufbau

Ein Laptop im Heimnetz verbindet sich mit dem Webserver 198.51.100.20 auf Port 443. Der Laptop hat die Adresse 192.168.1.10 und nutzt den Quellport 51000.

Im ersten Segment steht Seq = 1000, das Flag SYN und die Optionen MSS 1460, Window Scale 7, SACK Permitted und Timestamps. Der Server antwortet mit Seq = 5000, Ack = 1001, den Flags SYN und ACK und den passenden Optionen. Der Laptop bestätigt mit Seq = 1001 und Ack = 5001 und sendet anschließend die HTTP-Anfrage. Die Verbindung steht.

5.9 Beispiel 2: Keine Antwort auf das SYN

Angenommen, das Segment mit dem SYN erreicht den Server nicht, etwa weil ein Firewall-Eintrag es verwirft. Der Client wartet auf die Antwort und sendet das SYN nach einer Wartezeit erneut. Jeder weitere Versuch wartet länger als der vorige, die Wartezeit verdoppelt sich. Nach einer bestimmten Anzahl von Versuchen gibt das System auf und meldet der Anwendung einen Verbindungsfehler oder eine Zeitüberschreitung.

Dieses Verhalten erklärt, warum ein Webseitenaufruf bei einer blockierten Verbindung minutenlang hängen kann, bevor er scheitert.

5.10 Beispiel 3: Port nicht geöffnet

Angenommen, auf dem Server lauscht an Port 443 kein Dienst. Das SYN trifft ein, aber der Server hat keinen Socket dafür. Er antwortet mit einem Segment mit dem Flag RST und ACK. Der Client erfährt sofort, dass der Port geschlossen ist, und die Anwendung meldet einen Fehler wie Verbindung verweigert. Im Gegensatz zum vorigen Beispiel dauert die Fehlermeldung nur die Laufzeit einer Antwort.

Der Unterschied zwischen beiden Fällen ist ein guter Test. Eine sofortige Ablehnung deutet auf einen Dienst, der nicht läuft oder nicht lauscht. Eine lange Wartezeit deutet auf eine blockierte Verbindung hin.

5.11 Beispiel 4: Verbindungsaufbau mit Daten im Handshake

Die Anwendung kann nicht schon im SYN Daten mitschicken, außer bei speziellen Erweiterungen wie TCP Fast Open. Im normalen Ablauf beginnen die Daten erst nach dem dritten Segment. Das ist ein Grund, warum eine neue Verbindung eine volle Laufzeit extra kostet, bevor die erste Antwort kommt. Moderne Protokolle wie QUIC, das wir in Kapitel 13 besprechen, reduzieren diesen Aufwand.

> **Merke**
>
> Der Three-Way Handshake synchronisiert die Startnummern beider Seiten, bestätigt die Verbindung in beide Richtungen und handelt Optionen aus. Erst danach fließen Nutzdaten.

Schritt	Seq	Ack	Flags
1 Client	1000	0	SYN
2 Server	5000	1001	SYN, ACK
3 Client	1001	5001	ACK

Tabelle 5.1. Die drei Segmente des Handshakes im Beispiel. Die Werte im dritten Segment zeigen, dass beide Seiten die Startnummer der anderen bestätigt haben.

Kontrollfragen

1. Warum bestätigt der Server die ISN des Clients mit einem Wert, der um eins größer ist?
2. Warum genügen zwei Nachrichten nicht, um die Verbindung sicher aufzubauen?
3. Welche Option legt fest, wie groß die Nutzdaten eines einzelnen Segments höchstens sein dürfen?
4. Der Client bekommt sofort eine Fehlermeldung Verbindung verweigert. Was vermutest du, und was wäre bei einer langen Wartezeit anders?

Musterlösungen: 1. Das SYN verbraucht selbst eine Sequenznummer. Die Bestätigung nennt daher das nächste erwartete Byte, also ISN plus eins. 2. Der Server kann nach dem ersten SYN nicht wissen, ob seine Antwort ankommt. Erst das dritte Segment bestätigt beide Richtungen. 3. Die Option MSS. 4. Vermutlich ist an dem Port kein Dienst aktiv, der Server antwortet mit RST. Eine lange Wartezeit deutet dagegen auf eine blockierte oder verlorene Verbindung hin, weil keine Antwort zurückkommt.

KAPITEL 6

TCP-Datenübertragung

Nach dem Handshake fließen die Daten. Eine Webseite besteht aus vielen Kilobyte, die TCP in Segmente zerlegt und einzeln verschickt. Im Netz geht dabei etwas schief, und zwar regelmäßig: Ein Router verwirft ein Paket, weil sein Puffer voll ist, oder eine Leitung stört kurz. TCP muss diesen Verlust bemerken und ausgleichen, ohne dass die Anwendung etwas davon mitbekommt. In diesem Kapitel sehen wir, wie das geschieht, vom einfachen Fall bis zum Verlust mitten im Datenstrom.

6.1 Sequenznummern im Datenstrom

Jedes Byte der Nutzdaten hat eine Nummer. Ein Segment trägt die Sequenznummer seines ersten Bytes. Die folgenden Bytes des Segments zählen fortlaufend weiter. Sendet ein Client 3000 Byte in drei Segmenten zu je 1000 Byte und startet die Numerierung bei 1001, dann tragen die Segmente die Nummern 1001, 2001 und 3001. Das Byte 4000 ist das letzte des dritten Segments, die nächste Nummer wäre 4001.

Die Nummern machen zwei Dinge möglich. Der Empfänger kann die Segmente in die richtige Reihenfolge bringen, und er kann jede Lücke erkennen. Fehlt zwischen zwei bekannten Nummern ein Stück, weiß er genau, welche Bytes fehlen.

6.2 Datenübertragung in Segmenten

Die Anwendung schreibt Daten in einen Sendepuffer des Betriebssystems. TCP entnimmt daraus Stücke und verpackt sie in Segmente. Die Größe richtet sich nach der MSS, die wir im Handshake ausgehandelt haben. Ein Segment ist nie größer als die MSS, kann aber kleiner sein, wenn der Puffer nicht voll ist. Jedes Segment trägt eine Sequenznummer und eine Bestätigungsnummer.

6.3 Reihenfolge der Daten

Pakete können auf unterschiedlichen Wegen durch das Netz laufen. Ein Segment mit der Nummer 2001 kann vor dem Segment mit 1001 ankommen. TCP hält solche Segmente

im Empfangspuffer zurück und liefert die Daten erst in der richtigen Reihenfolge an die Anwendung aus. Die Anwendung sieht davon nichts, sie erhält einen lückenlosen Strom.

6.4 Empfangsbestätigungen

Der Empfänger bestätigt mit der Bestätigungsnummer das nächste Byte, das er erwartet. Angenommen, er hat die Bytes bis 3000 erhalten. Dann lautet seine Bestätigung 3001. Diese Angabe ist eindeutig: Alles bis 3000 ist angekommen, auf 3001 wird gewartet.

6.5 Cumulative Acknowledgments

TCP bestätigt kumulativ. Eine Bestätigung deckt also alle Bytes bis zu der genannten Nummer ab, nicht nur das letzte Segment. Ein ACK mit der Nummer 5001 bestätigt damit alle Bytes von 1 bis 5000 auf einmal. Das spart Bestätigungen, hat aber eine Schwäche: Ein einzelner verlorener Teil verhindert, dass die Bestätigung weiterwandert, auch wenn später Segmente angekommen sind.

6.6 Duplicate ACKs

Kommt ein Segment an, dessen Nummer nicht direkt an die bisher bestätigten Daten anschließt, wiederholt der Empfänger die alte Bestätigungsnummer. Diese Wiederholung heißt Duplicate ACK. Sie zeigt dem Absender: Ich habe etwas nicht bekommen, und ich warte weiter auf das Byte mit dieser Nummer. Erhält der Absender drei Duplicate ACKs mit derselben Nummer, schließt er daraus, dass das Segment verloren ist.

6.7 Out-of-Order-Segmente

Segmente, die nicht in der erwarteten Reihenfolge ankommen, heißen Out-of-Order-Segmente. Sie sind nicht fehlerhaft, nur in falscher Reihenfolge. Der Empfänger speichert sie zwischen, bestätigt aber weiterhin nur die lückenlosen Daten. Sobald die fehlende Lücke gefüllt ist, springt die Bestätigung über alle gespeicherten Segmente hinweg.

6.8 Retransmission

Eine Retransmission ist das erneute Senden eines Segments, das als verloren gilt. Der Absender speichert alle gesendeten, aber noch nicht bestätigten Segmente. Wird eines davon nicht rechtzeitig bestätigt, schickt er es noch einmal. Die Speicherung ist notwendig, weil der Absender die Daten sonst nicht erneut erzeugen könnte.

6.9 Timeout

Ein Timeout ist der Ablauf einer Wartezeit ohne Bestätigung. Der Absender misst die Laufzeit der Verbindung, also die Zeit zwischen dem Senden eines Segments und der Bestätigung. Daraus berechnet er die Retransmission Timeout, kurz RTO. Erreicht die Wartezeit diesen Wert, sendet er das Segment erneut. Die RTO ist nie kleiner als ein festgelegter Mindestwert, und nach jeder erfolglosen Wiederholung verdoppelt sie sich.

Der Timeout ist der letzte Ausweg. Er dauert meist länger als eine Fast Retransmission, weil der Absender lange warten muss, bevor er überhaupt handelt.

6.10 Fast Retransmit

Fast Retransmit ist eine Abkürzung. Hat der Absender drei Duplicate ACKs mit derselben Nummer erhalten, sendet er das fehlende Segment sofort neu, ohne den Timeout abzuwarten. Die Zahl drei verhindert, dass ein einzelnes vertauschtes Paket schon eine Wiederholung auslöst. Das Verfahren stammt aus RFC 5681 und ist heute Standard.

Die folgende Abbildung zeigt diesen Ablauf. Das zweite Segment geht auf dem Weg verloren. Die folgenden Segmente kommen an, lösen aber nur Duplicate ACKs aus. Nach dem dritten Duplicate ACK sendet der Client das fehlende Segment neu.

6.11 Selective Acknowledgment (SACK)

Die kumulative Bestätigung hat die Schwäche, dass sie nur eine Lücke benennen kann. Gehen mehrere Segmente verloren, muss der Absender oft raten, welche Teile fehlen. Mit der Option SACK kann der Empfänger genau mitteilen, welche Bereiche angekommen sind. Die Option enthält bis zu drei oder vier Bereiche mit Anfang und Ende. Der Absender sendet dann nur die tatsächlich fehlenden Teile neu. SACK ist in RFC 2018 beschrieben und wurde im Handshake ausgehandelt, wie wir im Kapitel 5 gesehen haben.

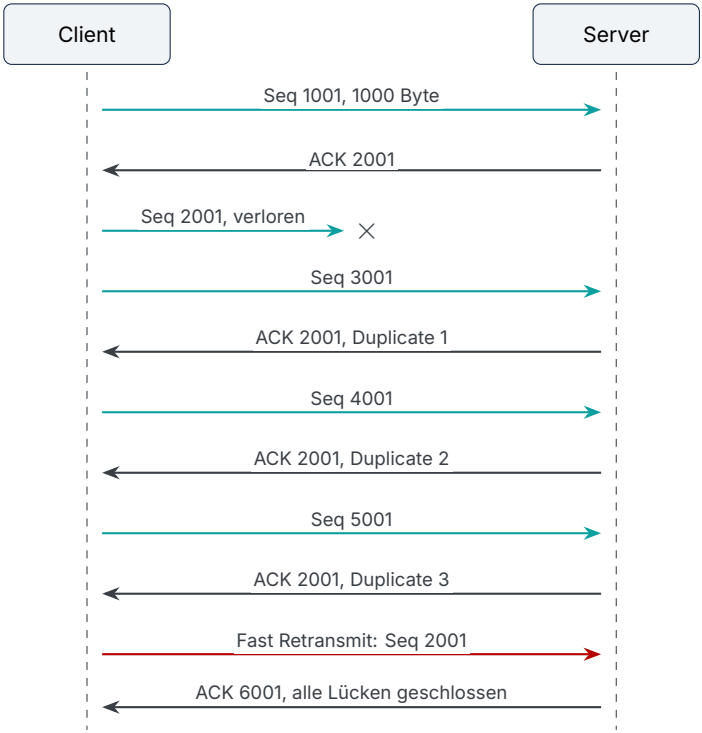


Abbildung 6.1. Verlust eines Segments und Fast Retransmit. Das Segment mit der Sequenznummer 2001 geht verloren. Die folgenden Segmente lösen Duplicate ACKs mit der Nummer 2001 aus. Nach dem dritten Duplicate ACK wird das fehlende Segment sofort neu gesendet. Schematische Darstellung.

6.12 TCP-Fehlerbehandlung im Überblick

TCP erkennt Verluste auf zwei Wegen: durch Duplicate ACKs und durch den Timeout. Die Duplicate ACKs liefern den schnelleren Hinweis, der Timeout ist die letzte Sicherung. Dazu kommt die Prüfsumme, die beschädigte Segmente verwirft. Ein verworfenes Segment wird wie ein verlorenes behandelt. Der Absender sieht den Effekt also in beiden Fällen gleich.

Ereignis	Reaktion des Absenders
Drei Duplicate ACKs	Fast Retransmit des fehlenden Segments
Timeout ohne ACK	Retransmission, RTO verdoppelt sich
SACK-Bereich gemeldet	nur die fehlenden Bereiche neu senden
Segment mit falscher Prüfsumme	wird vom Empfänger verworfen, wie ein Verlust behandelt

Tabelle 6.1. Die Reaktionen des Absenders auf verschiedene Ereignisse.

Merke

TCP bemerkt Verluste an fehlenden Bestätigungen. Drei Duplicate ACKs lösen eine sofortige Wiederholung aus, ein Timeout ist die letzte Rückfallebene. Die Empfangsbestätigung nennt immer das nächste erwartete Byte.

Kontrollfragen

1. Der Client sendet Segmente mit den Nummern 1001, 2001 und 3001 zu je 1000 Byte. Welche Bestätigungsnummer schickt der Server nach dem Empfang des ersten und dritten Segments, wenn das zweite fehlt?
2. Warum wartet TCP bei einem Duplicate ACK nicht gleich auf den Timeout?
3. Welchen Vorteil bringt SACK gegenüber der kumulativen Bestätigung?
4. Warum speichert der Absender gesendete Segmente, bis sie bestätigt sind?

Musterlösungen: 1. Nach dem ersten Segment lautet die Bestätigung 2001. Nach dem dritten bleibt sie bei 2001, weil das zweite Segment fehlt. Die Antwort wiederholt sich als Duplicate ACK. 2. Der Timeout ist lang, weil er die Laufzeit mit Sicherheitsreserve abwartet. Drei Duplicate ACKs zeigen den Verlust schneller an. 3. SACK nennt genau die angekommenen Bereiche, sodass nur die tatsächlich fehlenden Teile neu gesendet werden müssen. 4. Nur so kann der Absender ein verlorenes Segment erneut senden, ohne die Daten neu aus der Anwendung zu holen.

KAPITEL 7

Flow Control und Sliding Window

Ein Server kann Daten schneller senden, als ein Client sie verarbeiten kann. Stell dir vor, ein Lagerarbeiter bekommt Kisten schneller, als er sie auspacken kann. Irgendwann stapeln sich die Kisten auf dem Boden, und der Arbeiter muss die Lieferung bremsen. TCP kennt dieses Problem ebenfalls. Die Flusskontrolle sorgt dafür, dass der Sender nie mehr Daten schickt, als der Empfänger aufnehmen kann. Dieses Kapitel erklärt die Mechanismen, vom Empfangsfenster bis zum Zero Window.

7.1 Was ist Flow Control?

Flow Control schützt den Empfänger vor Überlastung. Der Empfänger hat einen Puffer, in dem ankommende Daten warten, bis die Anwendung sie abholt. Ist dieser Puffer voll, müssen weitere Daten abgewiesen werden. Die Flusskontrolle verhindert, dass es so weit kommt. Der Empfänger teilt dem Sender mit, wie viel Platz er hat, und der Sender richtet sich danach.

7.2 Warum Flusskontrolle benötigt wird

Ohne Flusskontrolle würde ein schneller Server einen langsamen Client überfluten. Die Pakete würden im Puffer verloren gehen, der Sender müsste sie erneut schicken, und die Übertragung würde zäh. Mit Flusskontrolle erhält der Empfänger nur so viele Daten, wie er verarbeiten kann. Das spart Wiederholungen und sorgt für stabile Übertragungen.

7.3 Receive Window

Das Receive Window ist der freie Platz im Empfangspuffer. Der Empfänger kennt es genau, denn er weiß, wie viele Bytes er gespeichert und noch nicht an die Anwendung übergeben hat. Diesen Wert trägt er im Feld Window Size jedes Segments ein, das er sendet. Der Sender darf dann höchstens so viele Bytes unbestätigt unterwegs haben, wie das Fenster angibt.

7.4 Advertised Window

Das Advertised Window ist der Wert, den der Empfänger in jeder Antwort mitteilt. Er wird als Fenstergröße angekündigt. Der Sender merkt sich diese Angabe und hält sich daran. Ändert sich der Platz im Puffer, schickt der Empfänger eine neue Angabe. Dadurch passt sich der Sender laufend an die Lage beim Empfänger an.

7.5 Sliding Window

Das Sliding Window ist das Prinzip, mit dem der Sender das Fenster verschiebt. Man kann es sich als Schiebefenster über dem Datenstrom vorstellen. Links vom Fenster liegen Daten, die bereits bestätigt sind. Innerhalb des Fensters liegen Daten, die gesendet, aber noch nicht bestätigt sind, oder die als Nächstes gesendet werden dürfen. Rechts vom Fenster liegen Daten, die noch warten müssen.

Mit jeder Bestätigung rückt die linke Kante nach rechts. Dadurch öffnet sich Platz für neue Daten, und das Fenster gleitet über den Strom. Die Abbildung zeigt diesen Zustand.

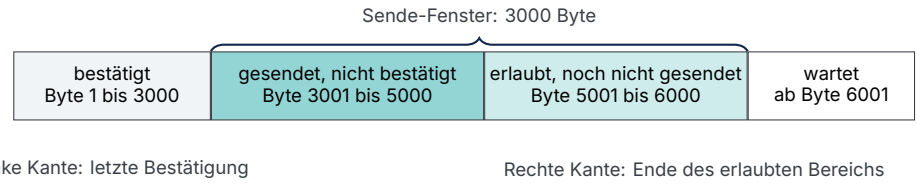


Abbildung 7.1. Das Sliding Window des Senders zu einem Zeitpunkt. Die Bytes links sind bestätigt. Das Fenster umfasst die Bytes, die gesendet oder als Nächstes erlaubt sind. Schematische Darstellung.

Das Fenster des Senders ist also nicht identisch mit dem Receive Window des Empfängers, aber es wird durch dessen Angabe begrenzt. Die Breite entspricht höchstens dem Advertised Window.

7.6 Window Size

Die Window Size ist das Feld im TCP-Header, das die Breite des Empfangsfensters trägt. Sie ist 16 Bit groß, und mit Window Scaling aus dem Handshake lässt sich der Wert vergrößern. Moderne Verbindungen über Netze mit großer Laufzeit brauchen so große Fenster, dass Window Scaling praktisch unverzichtbar ist.

Die Größe bestimmt den Durchsatz. Ein kleines Fenster bremst eine schnelle Verbindung, ein sehr großes füllt den Puffer nicht. Die optimale Größe hängt von der Laufzeit und der verfügbaren Bandbreite ab.

7.7 Zero Window

Ist der Empfangspuffer voll, meldet der Empfänger ein Fenster von null Byte. Diesen Zustand nennt man Zero Window. Der Sender darf dann keine Daten mehr senden, bis der Empfänger wieder Platz meldet.

Gehen dabei Angaben verloren, könnte der Sender ewig warten. Deshalb sendet er in regelmäßigen Abständen kleine Prüfsegmente, die Window Probes. Sie erkundigen sich nach dem aktuellen Stand. Sobald die Anwendung Daten abgeholt hat und der Puffer frei wird, schickt der Empfänger ein Window Update mit der neuen Größe. Die Übertragung läuft dann weiter.

7.8 Window Update

Ein Window Update ist eine Bestätigung ohne neue Daten, die nur die neue Fenstergröße mitteilt. Der Empfänger sendet es, sobald sich der freie Platz merklich vergrößert hat. Eine Schutzregel verhindert, dass winzige Änderungen ständig neue Segmente auslösen. Das spart Datenverkehr und vermeidet sogenannte Silly Window Syndrome, bei dem nur wenige Bytes pro Segment übertragen werden.

7.9 Buffer

Beide Seiten haben Puffer. Der Sendepuffer hält Daten, die noch nicht bestätigt sind, damit sie bei Verlust erneut gesendet werden können. Der Empfangspuffer hält ankommende Daten, bis sie vollständig und in der richtigen Reihenfolge sind und die Anwendung sie abholt. Die Größe beider Puffer begrenzt, wie viel Verkehr gleichzeitig möglich ist.

7.10 Zusammenspiel von Sender und Empfänger

Der Ablauf lässt sich als Kreislauf beschreiben. Der Sender sendet Daten innerhalb seines Fensters. Der Empfänger speichert sie, bestätigt die lückenlosen Teile und meldet

den freien Platz. Die Anwendung holt Daten ab, wodurch der Puffer leerer wird. Mit jeder Meldung rückt das Fenster weiter, und der Kreislauf beginnt von vorn.

> **Merke**
>
> Die Flusskontrolle begrenzt den Sender durch die Fenstergröße des Empfängers. Das Sliding Window verschiebt sich mit jeder Bestätigung, und ein Zero Window stoppt den Sender, bis der Empfänger wieder Platz hat.

Kontrollfragen

1. Der Empfänger meldet die Fenstergröße 0. Was tut der Sender, und welche Nachrichten schickt er?
2. Ein Sender hat 3000 Byte gesendet, aber noch nichts davon bestätigt bekommen. Das Fenster beträgt 6000 Byte. Wie viel darf er noch senden?
3. Warum ist Window Scaling für Verbindungen mit hoher Laufzeit wichtig?
4. Was unterscheidet Flusskontrolle von der Bestätigung der Daten?

Musterlösungen: 1. Der Sender stellt das Senden ein und schickt in Abständen Window Probes, bis der Empfänger ein Window Update mit einer Größe über null meldet. 2. Das Fenster umfasst 6000 Byte, davon sind 3000 unbestätigt unterwegs. Er darf also noch 3000 Byte senden. 3. Ein 16-Bit-Feld erlaubt höchstens 65535 Byte. Bei langen Laufzeiten braucht der Sender ein größeres Fenster, damit die Leitung ausgelastet bleibt. 4. Die Bestätigung meldet, welche Daten angekommen sind. Die Flusskontrolle meldet, wie viel Platz der Empfänger noch hat.

KAPITEL 8

TCP Congestion Control

Im letzten Kapitel ging es um einen Empfänger, der nicht mehr Daten aufnehmen kann. Das Problem ist aber nicht immer der Empfänger. Auch das Netz selbst kann überlastet sein: Viele Verbindungen teilen sich dieselbe Leitung oder denselben Router, dessen Puffer dann überläuft. Schickt jeder Sender ungebremst so viel er kann, bricht der Verkehr zusammen, und niemand kommt voran. TCP muss also zusätzlich herausfinden, wie viel das Netz verträgt. Dafür dient die Congestion Control, die Überlastkontrolle. Dieses Kapitel erklärt ihre Grundidee und die wichtigsten Verfahren.

8.1 Flow Control und Congestion Control im Vergleich

Beide Verfahren begrenzen den Sender, aber sie schützen vor verschiedenen Gefahren. Die Flusskontrolle schützt den Empfänger vor Überforderung und wird durch sein Receive Window bestimmt. Die Überlastkontrolle schützt das Netz vor zu vielen Daten und wird vom Sender selbst aus Verlusten und Laufzeiten geschätzt. Ein Sender darf also nur so viel senden, wie beide Grenzen zusammen erlauben. Die kleinere der beiden Größen gewinnt.

Ein Vergleich aus dem Verkehr hilft. Die Flusskontrolle entspricht der Frage, wie viele Autos das Ziel-Parkhaus aufnehmen kann. Die Überlastkontrolle entspricht der Frage, wie viele Autos die Autobahn bewältigt, bevor es zum Stau kommt.

8.2 Netzwerküberlastung

Eine Überlastung entsteht, wenn mehr Daten in einen Abschnitt des Netzes gelangen, als er weiterleiten kann. Router haben Puffer, die kurzfristig Spitzen auffangen. Sind die Puffer voll, verwerfen sie Pakete. Der Verlust ist also ein Zeichen für Überlastung, und TCP nutzt genau dieses Signal.

Außerdem zeigt eine wachsende Laufzeit, dass Daten in Warteschlangen stehen. Moderne Verfahren beachten auch das.

8.3 Congestion Window

Das Congestion Window, kurz cwnd, ist der Wert, den der Sender selbst festlegt. Er gibt an, wie viele Bytes er höchstens unbestätigt im Netz haben darf. Der Sender nutzt dabei den kleineren Wert aus Congestion Window und Receive Window. Das cwnd wächst, solange das Netz Daten gut weitergibt, und schrumpft, sobald es Verluste gibt.

Der Sender schätzt das cwnd also aus dem Verhalten des Netzes. Er muss nicht fragen, wie viel das Netz verträgt. Er tastet sich heran.

8.4 Slow Start

Am Anfang einer Verbindung kennt der Sender das Netz nicht. Deshalb beginnt er mit einem kleinen Fenster, dem Initial Window. Nach RFC 6928 beträgt es heute meist zehn Segmente. Mit jeder bestätigten Runde verdoppelt sich das Fenster, weil für jedes bestätigte Segment ein neues gesendet werden darf. Man nennt das exponentielles Wachstum, obwohl der Name Slow Start irreführend klingt.

Das Wachstum endet, wenn ein Verlust auftritt oder ein Schwellwert erreicht ist, den man Slow-Start-Threshold nennt.

8.5 Congestion Avoidance

Ab dem Schwellwert wechselt der Sender in die Congestion Avoidance. Das Fenster wächst nun langsamer, nämlich um etwa ein Segment pro Laufzeit. Der Sender tastet sich vorsichtig an die Grenze des Netzes heran, statt sie mit großen Sprüngen zu überschreiten.

Die folgende Abbildung zeigt den Verlauf des Congestion Window über die Zeit. Die Phasen sind deutlich zu erkennen: steiler Anstieg im Slow Start, flacher Anstieg in der Congestion Avoidance und ein Einbruch nach einem Verlust.

8.6 Packet Loss und Reaktion

Ein Verlust bedeutet für TCP ein Warnsignal. Bei einem Timeout nimmt der Sender an, dass das Netz stark überlastet ist, und setzt das cwnd auf einen sehr kleinen Wert zurück, meist ein Segment. Er beginnt dann wieder mit dem Slow Start. Bei Fast Retransmit, also drei Duplicate ACKs, ist das Signal milder, denn Daten fließen noch. Das Verfahren Reno halbiert das Fenster nur und setzt es mit der Fast Recovery fort.

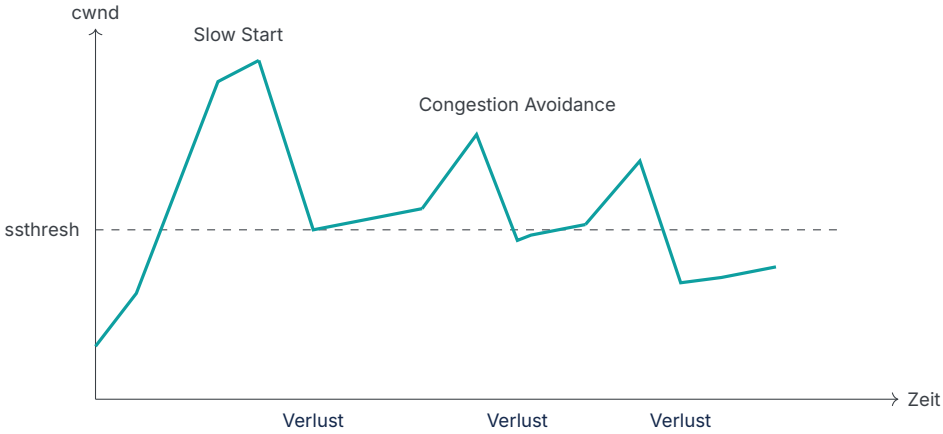


Abbildung 8.1. Verlauf des Congestion Window bei einem typischen TCP-Verfahren. Nach jedem Verlust sinkt das Fenster, dann wächst es wieder. Die Werte sind schematisch.

Diese Unterscheidung spielt im Alltag eine große Rolle. Ein Timeout bremst eine Verbindung spürbar stärker als eine einzelne Wiederholung.

8.7 RTT und RTO

Die Round Trip Time, kurz RTT, ist die Zeit von einem gesendeten Segment bis zu seiner Bestätigung. Sie schwankt, deshalb glättet TCP die Messwerte. Aus dem geglätteten Wert und einer Streuung berechnet der Sender die Retransmission Timeout, kurz RTO. RFC 6298 beschreibt die Berechnung und setzt einen Mindestwert von einer Sekunde. Der Wert passt sich laufend an die gemessenen Laufzeiten an.

RTT und RTO sind eng verwandt, aber nicht gleich. Die RTT ist eine Messung, die RTO ist die Grenze, ab der der Sender aufgibt, auf eine Bestätigung zu warten.

8.8 AIMD-Grundprinzip

Das Grundprinzip der klassischen Überlastkontrolle heißt AIMD, Additive Increase Multiplicative Decrease. Das bedeutet: Das Fenster wächst langsam und gleichmäßig, aber bei einem Verlust wird es stark verkleinert, meist halbiert. Dieses Muster erzeugt die

Sägezahnform in der Abbildung.

Die Stärke des Prinzips liegt in der Fairness. Wenn mehrere Verbindungen dieselbe Leitung teilen, senkt jede bei Verlust ihr Fenster deutlich. Dadurch teilen sich die Verbindungen die Kapazität über die Zeit annähernd gleich auf.

8.9 Historische und moderne Verfahren

TCP-Varianten unterscheiden sich in der Reaktion auf Verluste und in der Schätzung der Netzkapazität. Die folgende Tabelle gibt einen Überblick. Die Liste ist nicht vollständig, sie zeigt die wichtigsten Vertreter.

Verfahren	Zeitraum	Kennzeichen
Tahoe	um 1988	cwnd auf ein Segment nach jedem Verlust
Reno	um 1990	Fast Recovery, Halbierung bei drei Duplicate ACKs
NewReno	seit 1999	verbessert die Behandlung mehrerer Verluste im selben Fenster
CUBIC	seit 2008	kubische Fensterfunktion, Standard in Linux
BBR	seit 2016	misst Bandbreite und Laufzeit statt Verluste

Tabelle 8.1. Wichtige TCP-Verfahren zur Überlastkontrolle. Die Jahresangaben sind Richtwerte für die Verbreitung.

Die historischen Verfahren reagieren vor allem auf Verluste. Moderne Verfahren wie BBR schätzen dagegen die Engpassbandbreite und die Mindestlaufzeit direkt. Sie können deshalb auch dann schnell sein, wenn ein Pufferrand nur wenige Verluste erzeugt. Welches Verfahren ein System nutzt, lässt sich am Betriebssystem ablesen. Die Auswahl ist eine Einstellung des Systems und keine Eigenschaft des Protokolls.

Merke

Flusskontrolle schützt den Empfänger, Überlastkontrolle schützt das Netz. Der Sender darf höchstens so viel senden, wie beide Grenzen zusammen erlauben. Verluste sind das wichtigste Signal, auf das die klassischen Verfahren reagieren.

Kontrollfragen

1. Ein Sender hat ein Receive Window von 40 Segmenten und ein Congestion Window von 20 Segmenten. Wie viele Segmente darf er höchstens unbestätigt senden?
2. Warum wächst das Congestion Window im Slow Start schneller als in der Congestion Avoidance?
3. Was bedeutet die Abkürzung AIMD, und warum ist das Verfahren fair?
4. Welche Reaktion auf einen Verlust ist stärker, ein Timeout oder drei Duplicate ACKs, und warum?

Musterlösungen: 1. Der Sender nimmt das kleinere Fenster, also 20 Segmente. 2. Im Slow Start verdoppelt sich das Fenster pro Laufzeit, in der Congestion Avoidance wächst es nur um ein Segment pro Laufzeit. 3. AIMD steht für Additive Increase Multiplicative Decrease: langsames additives Wachsen und starkes multiplikatives Verkleinern. Alle Verbindungen auf einer Leitung senken bei Verlust ihr Fenster, daher teilen sie die Kapazität annähernd gleich. 4. Ein Timeout wirkt stärker, weil der Sender das Fenster auf ein Segment zurücksetzt und neu mit dem Slow Start beginnt. Bei drei Duplicate ACKs halbiert er das Fenster nur.

KAPITEL 9

TCP-Verbindungsabbau und Zustände

Eine TCP-Verbindung endet nicht einfach. Beide Seiten müssen sich verabschieden, Daten zu Ende übertragen und sicherstellen, dass alle letzten Nachrichten angekommen sind. Außerdem merkt sich jedes Gerät, in welchem Zustand sich eine Verbindung befindet. Diese Zustände erklären viele Beobachtungen im Netz, etwa warum nach einem Abbau noch Verbindungen in der Liste stehen. In diesem Kapitel betrachten wir den geordneten Abbau, den harten Abbruch mit RST und die Zustandsmaschine, die hinter allem steht.

9.1 Wie wird eine TCP-Verbindung beendet?

Der Abbau läuft in beide Richtungen getrennt. Eine Seite erklärt, keine Daten mehr zu senden, die andere Seite kann aber noch weiter Daten schicken. Das ähnelt einem Telefonat, bei dem einer sagt: Ich habe nichts mehr zu sagen, du darfst aber noch erzählen. Erst wenn beide Seiten fertig sind, ist die Verbindung beendet.

9.2 FIN und ACK

Das Flag FIN erklärt, dass eine Seite keine Daten mehr senden wird. Es verbraucht wie das SYN eine Sequenznummer. Der Empfänger bestätigt das FIN mit einem ACK. Danach ist die Richtung von dieser Seite zu ihm geschlossen, die andere Richtung bleibt offen, bis auch dort ein FIN kommt.

9.3 Four-Way Termination

Ein normaler Abbau besteht aus vier Segmenten, deshalb heißt er Four-Way Termination. Die Seite, die zuerst den Abbau einleitet, nennt man aktiv schließend. Die Gegenseite nennt man passiv schließend. Die folgende Abbildung zeigt den Ablauf.

Zwischen dem zweiten und dritten Segment kann die Anwendung auf dem Server noch Daten senden. Deshalb sind zwei getrennte Nachrichten nötig, obwohl der Server die Bestätigung schon früh schicken könnte.

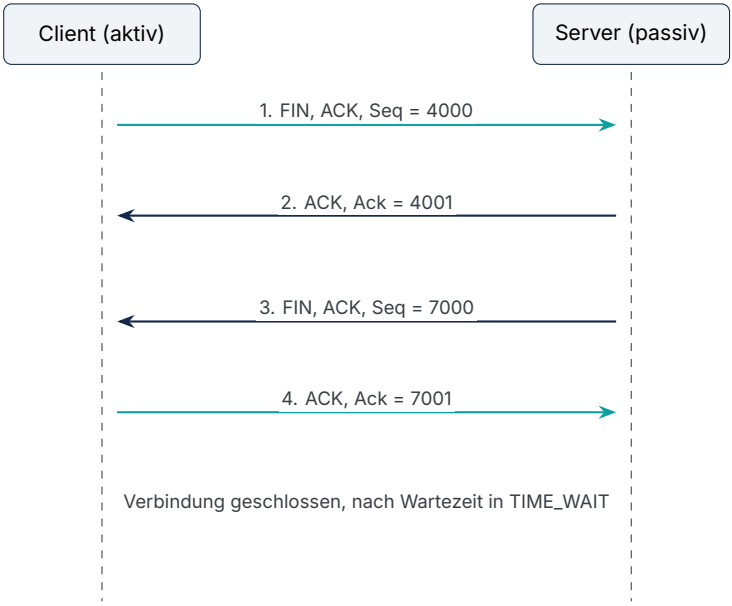


Abbildung 9.1. Der normale Verbindungsabbau in vier Schritten. Die Seite, die zuerst FIN sendet, bleibt nach dem Abbau im Zustand TIME_WAIT. Die Werte sind Beispiele. Schematische Darstellung.

9.4 Simultaneous Close

Manchmal senden beide Seiten gleichzeitig ein FIN. Das nennt man Simultaneous Close. Beide Seiten gehen dann in einen gemeinsamen Zwischenzustand, CLOSING, und bestätigen das jeweils andere FIN. Das Ergebnis ist ähnlich wie beim normalen Abbau, nur mit einer Nachricht weniger, weil die FIN-Segmente sich gegenseitig bestätigen.

9.5 RST

Das Flag RST bricht eine Verbindung sofort ab. Es ist kein geordneter Abbau, denn es verwirft alle noch ausstehenden Daten. Ein Gerät sendet RST, wenn an dem Port kein Dienst lauscht, wenn eine Verbindung unbekannt ist oder wenn die Anwendung hart beendet wird. Die Gegenseite erfährt den Abbruch sofort und meldet der Anwendung einen Fehler.

Die folgende Abbildung zeigt den Unterschied zum geordneten Abbau.



Abbildung 9.2. Verbindungsabbruch durch RST. Der Empfänger verwirft die Verbindung ohne den geordneten Abbau. Schematische Darstellung.

9.6 Zustände einer TCP-Verbindung

Jedes Gerät führt für jede Verbindung einen Zustand, der sich mit jedem Segment ändert. Diese Zustandsmaschine stammt aus RFC 793. Die folgende Tabelle erklärt die wichtigsten Zustände. Beim Verbindungsaufbau durchläuft man LISTEN, SYN_SENT, SYN_RECEIVED und ESTABLISHED. Beim Abbau kommen die anderen Zustände hinzu.

9.7 TIME_WAIT

TIME_WAIT ist der Zustand der aktiven Seite nach einem Abbau. Sie wartet doppelt so lange, wie ein Segment im Netz maximal unterwegs sein kann, also zweimal die Maximum Segment Lifetime, MSL. RFC 793 nennt dafür zwei Minuten als Richtwert. Der Grund: Ein spät angekommenes altes Segment soll nicht in eine neue Verbindung mit derselben Adressen- und Portkombination geraten. Die Wartezeit schützt also vor Verwechslungen.

Auf stark genutzten Servern sammeln sich viele TIME_WAIT-Einträge an. Das ist normal und begrenzt, belegt aber Ressourcen.

Zustand	Bedeutung
CLOSED	keine Verbindung, Ausgangs- und Endpunkt
LISTEN	Server wartet auf eingehende SYN
SYN_SENT	Client hat SYN gesendet und wartet auf SYN-ACK
SYN_RECEIVED	Server hat SYN empfangen und SYN-ACK gesendet
ESTABLISHED	Verbindung steht, Daten fließen
FIN_WAIT_1	aktive Seite hat FIN gesendet, wartet auf ACK
FIN_WAIT_2	FIN ist bestätigt, wartet auf das FIN der Gegenseite
CLOSE_WAIT	passive Seite hat FIN empfangen, Anwendung muss noch schließen
CLOSING	beide Seiten haben gleichzeitig FIN gesendet
LAST_ACK	passive Seite hat eigenes FIN gesendet, wartet auf ACK
TIME_WAIT	aktive Seite wartet nach dem Abbau, bis alte Segmente verfallen

Tabelle 9.1. Die Zustände einer TCP-Verbindung und ihre Bedeutung.

9.8 CLOSE_WAIT

CLOSE_WAIT zeigt an, dass die Gegenseite ihr FIN gesendet hat, die lokale Anwendung aber noch nicht geschlossen wurde. Der Zustand bleibt so lange bestehen, bis die Anwendung den Socket schließt. Viele CLOSE_WAIT-Einträge deuten deshalb meist auf einen Programmfehler hin, bei dem die Anwendung Verbindungen nicht schließt.

9.9 Weitere Zustände und Übergänge

Die übrigen Zustände folgen dem gleichen Muster. FIN_WAIT_1 und FIN_WAIT_2 gehören zur aktiven Seite, LAST_ACK zur passiven. Ein Zustand wechselt nur bei bestimmten

Segmenten. Wer die Übergänge kennt, kann aus einer Momentaufnahme erkennen, wo eine Verbindung hängt.

Mit dem Befehl ss -tan lässt sich der aktuelle Zustand aller TCP-Verbindungen auf Linux anzeigen. Unter Windows zeigt netstat -ano die Verbindungen mit Zustand an.

> **Merke**
>
> Ein geordneter Abbau braucht vier Segmente und läuft in beide Richtungen getrennt ab. RST bricht sofort ab. TIME_WAIT schützt die Verbindung vor verspäteten Segmenten, CLOSE_WAIT zeigt eine Anwendung, die nicht schließt.

Kontrollfragen

1. Warum bleibt die aktive Seite nach dem Abbau im Zustand TIME_WAIT?
2. Was bedeutet ein großer Bestand an CLOSE_WAIT-Verbindungen auf einem Server?
3. Worin unterscheidet sich ein RST von einem FIN?
4. Warum sind beim normalen Abbau zwei Nachrichten von der passiven Seite nötig?

Musterlösungen: 1. Die Wartezeit verhindert, dass verspätete Segmente der alten Verbindung in eine neue mit gleichen Adressen und Ports gelangen. 2. Die Anwendung schließt Verbindungen nach dem Empfang des FIN nicht. Das deutet meist auf einen Programmfehler hin. 3. FIN beendet eine Richtung geordnet und lässt Daten zu Ende übertragen. RST bricht die Verbindung sofort ab und verwirft ausstehende Daten. 4. Die passive Seite bestätigt das FIN zuerst, und erst wenn die Anwendung fertig ist, sendet sie ihr eigenes FIN. Dazwischen können noch Daten fließen.

KAPITEL 10

UDP: User Datagram Protocol

Nicht jede Nachricht braucht Sicherheit auf Kosten der Geschwindigkeit. Stell dir vor, du telefonierst über das Internet. Wenn ein Wort einer Sprachsequenz verloren geht, möchtest du nicht, dass die Verbindung anhält und das Wort nachgeliefert wird. Bis dahin hast du längst den nächsten Satz gehört, und die Nachlieferung wäre nur noch störend. Für solche Fälle gibt es UDP, das User Datagram Protocol. Es verzichtet bewusst auf die Mechanismen von TCP und bietet dafür Einfachheit und Tempo. Dieses Kapitel zeigt, was UDP leistet, was es nicht leistet und warum es dennoch unverzichtbar ist.

10.1 Was ist UDP?

UDP ist ein Transportprotokoll, das in RFC 768 beschrieben ist. Es erweitert IP um die Portnummern, damit Daten einer bestimmten Anwendung zugestellt werden können. Mehr tut es nicht. Der Name User Datagram Protocol beschreibt diese Schlichtheit: Ein Datagramm ist eine einzelne, eigenständige Nachricht.

Man kann UDP mit einem Postkarten-Versand vergleichen. Du schreibst die Adresse, den Absender und den Text auf die Karte und wirfst sie in den Briefkasten. Ob sie ankommt, erfährst du nicht, außer jemand antwortet dir. Es gibt keine Empfangsbestätigung, keine Nummerierung und keine Nachsendung.

10.2 Eigenschaften von UDP

UDP hat wenige Eigenschaften, die sich klar benennen lassen. Es ist verbindungslos, das heißt, es baut keine Verbindung auf. Jedes Datagramm ist unabhängig vom anderen. Es bietet keine eingebaute Zuverlässigkeit und keine Reihenfolge. Außerdem fehlt eine Flusskontrolle und eine Überlastkontrolle. Die einzige Sicherung ist eine optionale Prüfsumme über den Inhalt.

Gerade die Abwesenheit dieser Mechanismen macht UDP schnell und schlank. Es gibt keinen Handshake, der Zeit kostet, und keine Zustände, die Speicher binden.

10.3 Verbindungsloses Protokoll

Ein UDP-Sender schickt einfach los. Er muss nicht prüfen, ob der Empfänger bereit ist, und er muss nicht wissen, ob ein Dienst lauscht. Erst wenn ein Datagramm an einen Port ohne Dienst gelangt, kann der Empfänger mit einer ICMP-Meldung Port Unreachable antworten. Das ist eine indirekte Rückmeldung und keine Bestätigung der Zustellung.

Für den Empfänger gilt dasselbe. Er bekommt Datagramme, ohne dass ein Verbindungsaufbau stattgefunden hat. Eine Anwendung öffnet einen UDP-Socket an einem Port und empfängt, was dort ankommt.

10.4 Datagramme

Die Dateneinheit von UDP heißt Datagramm. Jedes Datagramm enthält einen Header von acht Byte und die Nutzdaten der Anwendung. Ein Datagramm bleibt beim Versand eine Einheit. Der Empfänger bekommt entweder das vollständige Datagramm oder gar nichts. Eine Anwendung, die mit UDP arbeitet, muss daher selbst bestimmen, wie groß ihre Nachrichten sind.

10.5 Keine Empfangsbestätigung durch UDP selbst

UDP bestätigt nichts. Ein gesendetes Datagramm löst keine ACK aus. Wenn eine Antwort gewünscht ist, muss die Anwendung sie selbst vorsehen, wie bei DNS. Der Client schickt eine Anfrage, und die Antwort des Servers gilt als Bestätigung. Fehlt sie, wiederholt die Anwendung die Anfrage, nicht UDP.

10.6 Keine eingebaute Wiederholungsübertragung

Verlorene Datagramme werden von UDP nicht wiederholt. Ist die Wiederholung wichtig, übernimmt die Anwendung diese Aufgabe. Das geschieht zum Beispiel bei DNS, wo ein Client nach einer Wartezeit erneut fragt. Bei einem Videostream dagegen wäre eine Wiederholung oft sinnlos, weil das Bild längst weitergelaufen ist.

10.7 Keine eingebaute Flusskontrolle

UDP prüft nicht, ob der Empfänger Daten verarbeiten kann. Ein schneller Sender kann einen langsamen Empfänger daher überfluten, und die Datagramme werden im Puffer verworfen. Auch die Überlastkontrolle fehlt. Anwendungen, die viel über UDP senden, müssen daher selbst für ein vernünftiges Tempo sorgen. Viele moderne Verfahren wie QUIC bauen diese Mechanismen auf UDP neu auf, wie Kapitel 13 zeigt.

10.8 UDP-Checksum

Die Checksum im UDP-Header prüft Header und Nutzdaten auf Übertragungsfehler. Sie wird wie bei TCP über einen Pseudoheader mit Adressen aus dem IP-Header berechnet. Bei IPv4 ist sie optional: Ein Wert von null bedeutet, dass keine Prüfsumme berechnet wurde. Bei IPv6 ist sie dagegen Pflicht, weil IPv6-Router keine Prüfsumme im IP-Header mehr führen. Ein fehlerhaftes Datagramm wird verworfen, ohne dass der Absender es erfährt.

10.9 Vorteile und Nachteile

Die Vorteile von UDP sind klar: geringer Overhead, kein Verbindungsaufbau, keine Wartezeit auf Bestätigungen und volle Kontrolle durch die Anwendung. Die Nachteile sind ebenso deutlich. Verluste, Duplikate und Reihenfolgefehler muss die Anwendung selbst behandeln, falls sie überhaupt relevant sind. Ein Datagramm kann außerdem so groß werden, dass es fragmentiert werden muss, was Verluste wahrscheinlicher macht.

10.10 Wann wird UDP verwendet?

UDP eignet sich besonders dort, wo Aktualität wichtiger ist als Vollständigkeit. Dazu gehören Sprach- und Videoübertragungen, Online-Spiele, Live-Streams und Messwerte, die regelmäßig neu gesendet werden. Außerdem nutzen Dienste mit kurzen Anfragen UDP, weil ein Handshake sich nicht lohnt, etwa DNS, DHCP und NTP. Die Auswahl ist eine Entscheidung der Anwendung, nicht des Netzes.

Merke

UDP überträgt unabhängige Datagramme ohne Verbindung, ohne Bestätigung und ohne Wiederholung. Es ist schnell und schlank, und alle Zuverlässigkeit muss die Anwendung selbst leisten, falls sie gebraucht wird.

Kontrollfragen

1. Nenne zwei Eigenschaften, die UDP mit TCP teilt, und zwei, die es nicht hat.
2. Warum ist die UDP-Prüfsumme bei IPv6 Pflicht?
3. Eine Anwendung verwendet UDP für DNS. Wie erfährt sie, dass eine Anfrage verloren ging?
4. Warum wäre eine Wiederholung eines verlorenen Videopakets oft sinnlos?

Musterlösungen: 1. Gemeinsam sind die Portnummern und die Prüfsumme. Fehlen tun der Verbindungsaufbau, die Zuverlässigkeit, die Reihenfolge und die Flusskontrolle. 2. IPv6-Router führen keine Prüfsumme im IP-Header mehr. Die UDP-Prüfsumme ist daher die einzige Absicherung der Nutzdaten. 3. Die Anwendung wartet auf die Antwort und wiederholt die Anfrage nach einer Wartezeit, wenn keine kommt. 4. Bei Video ist ein verspätetes Bild nicht mehr nützlich. Eine Wiederholung würde nur Bandbreite kosten und die Verzögerung erhöhen.

KAPITEL 11

Der UDP-Header und die UDP-Datenübertragung

Der UDP-Header ist der kürzeste Header im Transportbereich. Mit nur acht Byte transportiert er alles, was UDP braucht. Diese Schlichtheit ist kein Zufall, sie folgt direkt aus dem Verzicht auf Verbindung und Sicherung. In diesem Kapitel betrachten wir die vier Felder, den Aufbau eines Datagramms über IPv4 und IPv6, die Besonderheiten der Prüfsumme und den Umgang mit Fragmentierung und Verlust.

11.1 Der Aufbau eines UDP-Datagramms

Ein UDP-Datagramm besteht aus dem Header und den Nutzdaten. Der Header hat vier Felder zu je zwei Byte. Zusammen ergeben sie acht Byte, unabhängig davon, wie groß die Nutzdaten sind. Die folgende Abbildung zeigt die Felder in zwei Zeilen zu je 32 Bit.

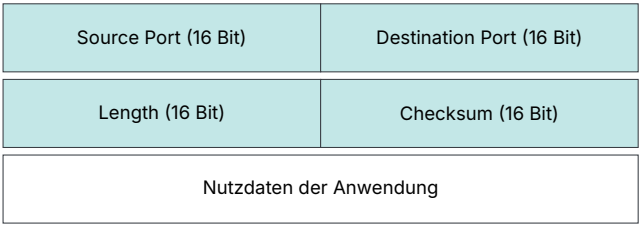


Abbildung 11.1. Der UDP-Header mit seinen vier Feldern in zwei Zeilen zu 32 Bit, gefolgt von den Nutzdaten. Schematische Darstellung.

11.2 Source Port

Der Source Port bezeichnet die Anwendung auf der Absenderseite. Er ist optional, ein Wert von null bedeutet, dass kein Port angegeben ist. Das ist vor allem dann sinnvoll, wenn keine Antwort erwartet wird. Bei Anfragen mit Antwort wie bei DNS ist der Quellport

dagegen wichtig, damit die Antwort zur richtigen Anwendung zurückfindet.

11.3 Destination Port

Der Destination Port bezeichnet den Dienst beim Empfänger, zum Beispiel 53 für DNS, 123 für NTP oder 67 für DHCP. Er entscheidet, welcher Socket die Datagramme bekommt. Trifft ein Datagramm auf einen Port ohne Socket, verwirft das System es und sendet im Normalfall eine ICMP-Meldung zurück.

11.4 Length

Das Feld Length gibt die Länge des gesamten Datagramms an, also Header und Nutzdaten zusammen, in Byte. Der kleinste Wert ist acht, denn ein Datagramm ohne Nutzdaten hat genau diese Länge. Der größte Wert ergibt sich aus der IP-Größe. Bei IPv4 sind das maximal 65507 Byte Nutzdaten, weil 65535 Byte Gesamtlänge minus 20 Byte IP-Header und 8 Byte UDP-Header bleiben.

11.5 Checksum

Die Checksum schützt den Header und die Nutzdaten vor Übertragungsfehlern. Sie wird über einen Pseudoheader berechnet, der Quell- und Zieladresse aus dem IP-Header enthält. Bei IPv4 ist die Prüfsumme optional. Ein Wert von null heißt, dass keine Prüfsumme berechnet wurde, und der Empfänger prüft dann nichts. Bei IPv6 ist die Prüfsumme dagegen verpflichtend, weil der IPv6-Header keine eigene Prüfsumme mehr trägt.

11.6 UDP über IPv4

Über IPv4 trägt das IP-Paket im Feld Protocol den Wert 17. Es folgt der UDP-Header und die Nutzdaten. Die Gesamtlänge des IP-Pakets ergibt sich aus dem IP-Header plus dem UDP-Datagramm. Da der IP-Header mindestens 20 Byte groß ist, bleibt für die Nutzdaten ein Platz von 65507 Byte.

11.7 UDP über IPv6

Über IPv6 steht im Next-Header-Feld ebenfalls der Wert 17, und der UDP-Header folgt direkt dem IPv6-Header, sofern keine Erweiterungsheader dazwischen liegen. Die Prüfsumme ist, wie beschrieben, Pflicht. Die Adressen im Pseudoheader sind 128 Bit lang.

11.8 Checksum-Besonderheiten

Bei der Berechnung der Prüfsumme gibt es einige Sonderfälle. Ist das Ergebnis null, wird als Wert 65535 eingetragen, damit die Null für den Verzicht reserviert bleibt. Bei IPv4 wird der Verzicht auf Prüfsummen von manchen Netzen nicht mehr akzeptiert, weil Fehler sonst unbemerkt bleiben. Für das Verständnis genügt es, sich zu merken, dass die Prüfsumme bei IPv4 optional und bei IPv6 Pflicht ist.

11.9 Fragmentierung und MTU

Ein UDP-Datagramm kann größer sein als die MTU des Netzes. Dann wird das IP-Paket fragmentiert, also in mehrere Teile zerlegt. Verliert ein Teil auf dem Weg, geht das gesamte Datagramm verloren, weil der Empfänger es nicht zusammensetzen kann. UDP selbst bekommt davon nichts mit und sendet nichts erneut.

Deshalb halten Anwendungen ihre Datagramme möglichst klein. Ein Wert von etwa 1472 Byte Nutzdaten ist bei Ethernet üblich, weil 1500 Byte MTU abzüglich 20 Byte IPv4-Header und 8 Byte UDP-Header genau diesen Platz lassen. Bei IPv6 ist die Grenze durch den 40 Byte großen Header kleiner.

11.10 Umgang mit Paketverlust

Paketverlust trifft UDP ungeschützt. Die Anwendung muss entscheiden, ob sie verlorene Datagramme ersetzt, ignoriert oder neu anfordert. Ein Sprachprogramm ignoriert Verluste oft und verdeckt sie durch Interpolation, also durch eine geschätzte Zwischenstufe. Ein Messwertsystem sendet die jeweils neueste Messung und verwirft ältere Werte. Eine DNS-Anwendung wiederholt die Anfrage nach einer Wartezeit.

11.11 Datagrammübertragung

Der Ablauf einer UDP-Übertragung ist schnell beschrieben. Der Client schickt ein Datagramm, der Server empfängt es und verarbeitet es. Antwortet er, geht ein weiteres Datagramm zurück. Ohne Antwort bleibt der Client ohne Information. Die Abbildung zeigt zwei Fälle: eine Anfrage mit Antwort und eine Anfrage, deren Datagramm verloren geht.



Abbildung 11.2. Zwei Fälle einer UDP-Anfrage. Im ersten Fall kommt die Antwort an. Im zweiten Fall geht die Anfrage verloren, UDP wiederholt sie nicht, die Anwendung muss das übernehmen. Schematische Darstellung.

> **Merke**
>
> Der UDP-Header ist acht Byte groß und besteht aus Ports, Länge und Prüfsumme. Die Prüfsumme ist bei IPv4 optional und bei IPv6 Pflicht. Große Datagramme werden fragmentiert, und dann geht bei einem Verlust das gesamte Datagramm verloren.

Kontrollfragen

1. Wie groß ist der UDP-Header, und welche vier Felder enthält er?
2. Welcher Wert im Checksum-Feld bedeutet bei IPv4, dass keine Prüfsumme berechnet wurde?

3. Warum sollten Anwendungen ihre UDP-Datagramme klein halten?
4. Was passiert, wenn ein Fragment eines großen UDP-Datagramms verloren geht?

Musterlösungen: 1. Acht Byte, mit Source Port, Destination Port, Length und Checksum. 2. Der Wert null. 3. Große Datagramme werden fragmentiert, und schon ein verlorenes Fragment macht das ganze Datagramm unbrauchbar. 4. Der Empfänger kann das Datagramm nicht zusammensetzen und verwirft alle Fragmente. UDP sendet nichts erneut, die Anwendung muss den Verlust bemerken.

KAPITEL 12

TCP und UDP im direkten Vergleich

Viele Fragen zu Netzwerken beginnen mit der Wahl des Transportprotokolls. Soll eine Anwendung TCP oder UDP nutzen? Die Antwort lautet selten ausschließlich das eine oder das andere. Sie hängt davon ab, was die Anwendung braucht: vollständige Daten, geringe Verzögerung, einfache Anfragen oder dauerhafte Verbindungen. Dieses Kapitel stellt beide Protokolle gegenüber und zeigt anhand typischer Anwendungen, wie man entscheidet.

12.1 Verbindungsorientiert versus verbindungslos

TCP baut vor dem Datenaustausch eine Verbindung auf, hält sie offen und baut sie am Ende geordnet ab. UDP sendet Datagramme ohne Verbindung. Der Unterschied bestimmt, wie viel Zustand beide Seiten speichern müssen und wie schnell die erste Antwort kommt. Bei TCP kostet der Aufbau eine volle Laufzeit, bei UDP sofort nichts.

12.2 Zuverlässigkeit und Reihenfolge

TCP garantiert, dass alle Daten vollständig und in der richtigen Reihenfolge ankommen, sofern die Verbindung besteht. UDP garantiert nichts davon. Ein Datagramm kann verloren gehen, doppelt ankommen oder in falscher Reihenfolge eintreffen. Ob das ein Problem ist, entscheidet die Anwendung. Für eine Datei ist jeder Verlust fatal, für einen Sprachkanal meist nicht.

12.3 Geschwindigkeit und Latenz

Die Latenz ist die Verzögerung bis zur Zustellung. TCP fügt durch Verbindungsaufbau, Bestätigungen und Wiederholungen Verzögerung hinzu. Ein verlorenes Paket blockiert bei TCP alle nachfolgenden Daten, bis es nachgeliefert ist. Das nennt man Head-of-Line Blocking, und es betrifft die Reihenfolge, nicht die Geschwindigkeit der Leitung. UDP liefert jedes Datagramm sofort aus, auch wenn ein früheres fehlt.

12.4 Headergröße und Overhead

Der TCP-Header ist mindestens 20 Byte groß, mit Optionen bis zu 60 Byte. Der UDP-Header hat acht Byte. Bei kleinen Nachrichten fällt dieser Unterschied stark ins Gewicht. Eine DNS-Anfrage mit 40 Byte Nutzdaten ergibt mit UDP ein IP-Paket von 68 Byte. Über TCP kommen 2 Byte Längenangabe, 20 Byte TCP-Header und 20 Byte IP-Header dazu, also 82 Byte, bevor Handshake und Bestätigungen hinzukommen.

12.5 Fehlerbehandlung

TCP erkennt Fehler, bestätigt Daten und wiederholt verlorene Segmente, ohne dass die Anwendung etwas davon merkt. UDP erkennt Fehler nur über die Prüfsumme und verwirft fehlerhafte Datagramme. Alles Weitere muss die Anwendung leisten. Diese Verteilung der Verantwortung ist ein zentraler Unterschied.

12.6 Flow Control und Congestion Control

TCP hat beide Mechanismen eingebaut, wie wir in den Kapiteln 7 und 8 gesehen haben. UDP hat keinen von beiden. Anwendungen, die über UDP viel senden, können das Netz überlasten und müssen selbst für eine angemessene Senderate sorgen. Das ist der Grund, warum UDP-Verkehr im Netz oft kritisch betrachtet wird, obwohl das Protokoll selbst völlig neutral ist.

12.7 Typische Einsatzgebiete

Die folgende Tabelle ordnet typische Anwendungen den Protokollen zu. Sie zeigt, dass beide Protokolle breit genutzt werden und dass die Wahl sich nach den Anforderungen richtet.

12.8 Warum nicht immer dasselbe Protokoll?

Eine Anwendung wählt nicht frei, sondern nach ihren Zielen. Eine Webseite braucht jede Zeile Text, daher ist TCP die richtige Wahl. Ein Videotelefonat braucht das aktuelle Bild, daher ist UDP besser. Beide Protokolle lösen unterschiedliche Probleme, und keines ist besser oder schlechter an sich.

Dazu kommt ein praktischer Grund. Viele Anwendungen wählen heute beides. Ein Videoclient nutzt TCP für die Steuerung und UDP für die Medien. QUIC wiederum läuft auf UDP und liefert trotzdem zuverlässige Datenströme. Die Grenze zwischen beiden Welten ist daher weniger scharf, als sie auf den ersten Blick wirkt.

12.9 Entscheidungshilfe

Die folgende Abbildung fasst die Entscheidung in wenigen Fragen zusammen. Sie ist eine Hilfe, keine feste Regel, denn die Praxis kennt viele Ausnahmen.

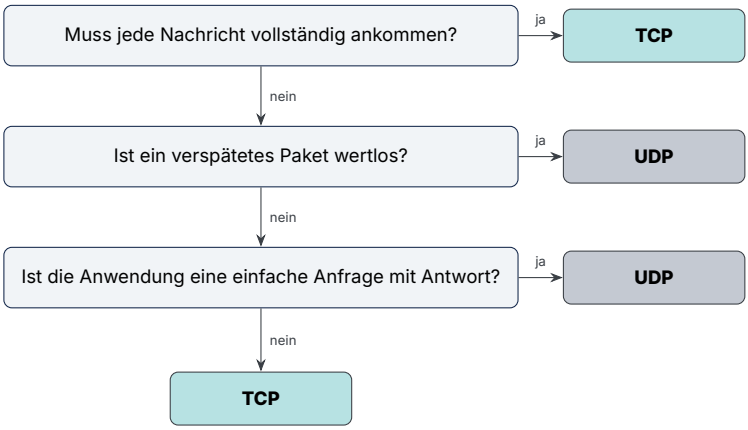


Abbildung 12.1. Entscheidungshilfe für die Wahl zwischen TCP und UDP. Die Fragen sind Anhaltspunkte, die Praxis kennt Ausnahmen. Schematische Darstellung.

> **Merke**
>
> TCP liefert Vollständigkeit und Reihenfolge, UDP liefert Tempo und Einfachheit. Die Anwendung entscheidet, was wichtiger ist, und viele Anwendungen kombinieren beide Protokolle.

Kontrollfragen

1. Warum ist TCP für SSH besser geeignet als UDP?
2. Eine Anwendung soll Sensordaten alle zehn Millisekunden übertragen, alte Werte sind nach kurzer Zeit wertlos. Welches Protokoll würdest du wählen, und warum?
3. Welche Verzögerung erzeugt das Head-of-Line Blocking bei TCP?
4. Warum ist DNS ein Sonderfall, der beide Protokolle nutzt?

Musterlösungen: 1. SSH überträgt Befehle und Ausgaben, die vollständig und in richtiger Reihenfolge ankommen müssen. TCP garantiert das. 2. UDP, weil alte Werte wertlos sind und eine Wiederholung nur Verzögerung bringt. Die Anwendung kann verlorene Werte einfach durch den nächsten ersetzen. 3. Ein verlorenes Paket hält alle nachfolgenden Daten zurück, bis es nachgeliefert ist, obwohl die bereits angekommenen Daten vorhanden sind. Das verzögert die Anwendung. 4. DNS-Anfragen sind kurz und passen in UDP. Große Antworten, etwa bei Zonenübertragungen, laufen über TCP, weil sie vollständig sein müssen.

Anwendung	Protokoll	Begründung
Webseiten (HTTP/1.1, HTTP/2)	TCP	vollständige Daten nötig
E-Mail (SMTP, IMAP)	TCP	keine verlorenen Nachrichten
Dateiübertragung	TCP	jedes Byte zählt
DNS	UDP und TCP	kurze Anfragen, große Antworten per TCP
DHCP	UDP	Anfrage vor der eigenen Adresse
VoIP und Telefonie	UDP	Verzögerung ist schädlicher als Verlust
Videokonferenzen	UDP	aktuelles Bild wichtiger als alte Bilder
Online-Gaming	UDP	neueste Position zählt
Streaming	beides	Live meist UDP, angepasste Dienste oft TCP
SSH	TCP	Befehle müssen vollständig ankommen
NTP	UDP	kurze Anfrage, Wiederholung nicht nötig

Tabelle 12.1. Typische Anwendungen und das Protokoll, das sie in der Regel nutzen.

KAPITEL 13

Moderne Transportprotokolle und QUIC

Bisher standen TCP und UDP nebeneinander, und jedes hatte seine Stärken. Doch das Internet hat sich verändert. Webseiten bestehen heute aus Dutzenden Ressourcen, die parallel geladen werden, und fast alles läuft verschlüsselt. Die klassische Kombination aus TCP und TLS, auf der HTTP/1.1 und HTTP/2 aufsetzen, erzeugt dabei zusätzliche Wartezeiten. QUIC ist ein moderner Transport, der diese Probleme angeht. Dieses Kapitel erklärt, warum er entwickelt wurde, wie er arbeitet und warum er nicht einfach ein anderer Name für UDP ist.

13.1 Was ist QUIC?

QUIC ist ein verschlüsseltes, zuverlässiges Transportprotokoll, das auf UDP aufsetzt. Es wurde ursprünglich bei Google entwickelt und ist seit 2021 als IETF-Standard festgelegt. Die Spezifikation besteht aus mehreren RFCs: RFC 9000 beschreibt den Transport, RFC 9001 die Einbindung von TLS, RFC 9002 die Erkennung und Behebung von Verlusten. Der Name steht für Quick UDP Internet Connections, die Schreibweise hat sich aber als eigenständiger Begriff etabliert.

13.2 Warum wurde QUIC entwickelt?

Es gibt drei Gründe. Der erste ist die Zeit für den Verbindungsaufbau. Bei TCP mit TLS 1.2 vergehen zwei Runden, bevor die Anwendung Daten senden darf, und zusätzlich der TCP-Handshake. Der zweite ist das Head-of-Line Blocking, das bei TCP alle Daten einer Verbindung hinter einem verlorenen Paket anhält. Der dritte Grund ist die Weiterentwicklung im Netz selbst. TCP ist in vielen Geräten fest verankert, und Änderungen dort dauern Jahrzehnte. Über UDP lässt sich ein neues Protokoll dagegen in der Anwendung ausliefern, ohne auf die Netzgeräte zu warten.

13.3 QUIC und UDP

QUIC verwendet UDP als Träger. Die Datagramme werden also von UDP transportiert, das Netz sieht sie als UDP-Verkehr, meist auf Port 443. Die eigentliche Zuverlässigkeit, die Reihenfolge und die Überlastkontrolle implementiert QUIC selbst, in der Anwendung oder im Betriebssystem. Die folgende Abbildung zeigt den Stapel.

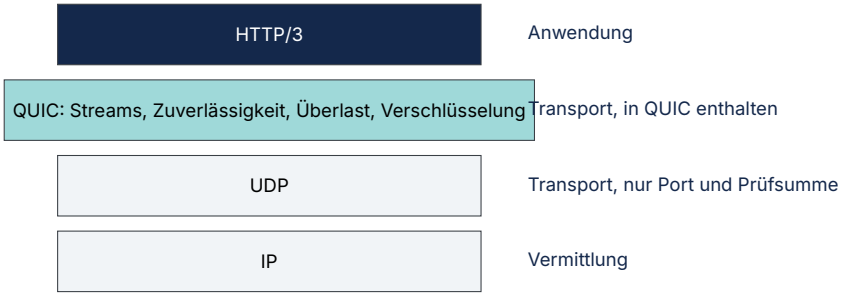


Abbildung 13.1. Der Protokollstapel von HTTP/3. QUIC übernimmt die Aufgaben, die bei TCP der Kern des Transports sind, und läuft dabei über UDP. Schematische Darstellung.

13.4 HTTP/3

HTTP/3 ist die Version von HTTP, die auf QUIC aufsetzt. Sie verwendet die Konzepte von HTTP/2, etwa mehrere parallele Anfragen auf einer Verbindung, transportiert sie aber über QUIC statt über TCP. Dadurch entfällt das Problem, dass ein verlorenes Paket alle Anfragen blockiert.

13.5 TLS-Integration

Bei TCP wird TLS als eigene Schicht darübergelegt. Bei QUIC ist TLS 1.3 fest eingebaut. Der Verbindungsaufbau und die Verschlüsselung laufen daher in einem Schritt. Fast alle Teile des QUIC-Headers und der Nutzdaten sind verschlüsselt, auch die Paketnummern. Ein Beobachter im Netz sieht nur wenige Angaben im Klartext.

13.6 Verbindungsaufbau

Eine neue QUIC-Verbindung benötigt bei TLS 1.3 nur eine Hin- und Rückfahrt, eine Runde, bevor Anfragen fließen können. Mit TCP und TLS 1.3 sind es zwei Runden. Mit TCP und TLS 1.2 sind es drei. Bei einer erneuten Verbindung zum selben Server erlaubt QUIC sogar 0-RTT, bei dem Anfragen schon im ersten Paket mitgeschickt werden. Dieser Vorteil hat eine Einschränkung: Die Daten im ersten Paket können wiederholt beim Server ankommen, deshalb sind sie nur für unkritische Anfragen geeignet.

Die folgende Abbildung vergleicht die Abläufe.



Schematische Darstellung, nicht maßstabsgetreu

Abbildung 13.2. Vergleich der Verbindungsaufbauten. TCP mit TLS benötigt mehrere Schritte, bevor die Anwendung senden darf. QUIC kombiniert Transport- und Verschlüsselungsaufbau in einer Runde.

13.7 Multiplexing

Eine QUIC-Verbindung kann viele Streams gleichzeitig führen. Ein Stream ist ein unabhängiger Datenstrom innerhalb der Verbindung, etwa für eine Webseite, ein Bild oder eine Schriftart. Jeder Stream hat eine eigene Nummerierung und eigene Bestätigungen. Fällt ein Paket aus, betrifft der Verlust nur den Stream, zu dem die Daten gehören. Die anderen Streams laufen weiter.

13.8 Head-of-Line Blocking

Das Head-of-Line Blocking entsteht, wenn ein verlorenes Paket alle nachfolgenden Daten zurückhält. Bei TCP betrifft das die gesamte Verbindung, weil TCP nur einen Strom kennt. Bei QUIC betrifft es nur den betroffenen Stream. Ein Bild kann so ausgeliefert werden, während auf die Schriftart noch gewartet wird. Das Problem verschwindet damit nicht vollständig, aber es wird auf das Nötigste begrenzt.

13.9 Connection Migration

Eine QUIC-Verbindung wird nicht über die IP-Adresse und den Port identifiziert, sondern über eine Verbindungskennung, die Connection ID. Wechselt ein Smartphone von WLAN zu Mobilfunk, ändern sich Adresse und Port, die Verbindung kann aber mit derselben Kennung fortgesetzt werden. Bei TCP bricht die Verbindung in diesem Fall meist ab, weil die Vierergruppe aus Adressen und Ports nicht mehr passt.

13.10 TCP versus QUIC

Die folgende Tabelle stellt die wichtigsten Unterschiede gegenüber. QUIC hat Vorteile, bringt aber auch Nachteile, die man kennen sollte.

Merkmal	TCP mit TLS	QUIC
Basis	TCP	UDP
Verschlüsselung	eigene TLS-Schicht	fest eingebaut
Verbindungsaufbau	2 bis 3 Runden	1 Runde, 0-RTT möglich
Head-of-Line Blocking	ganze Verbindung	nur pro Stream
Migration	meist Abbruch	über Connection ID
Sichtbarkeit für Netzgeräte	Header lesbar	weitgehend verschlüsselt

Tabelle 13.1. Gegenüberstellung von TCP mit TLS und QUIC.

Die Verschlüsselung hat eine Schattenseite. Netzbetreiber können den Verkehr nicht mehr so leicht analysieren oder filtern wie bei TCP. Das erschwert die Fehlersuche, und manche Firewalls müssen sich umstellen. Außerdem blockieren einige Netze UDP auf Port 443 bewusst, damit Anwendungen auf TCP zurückfallen. Browser erkennen das und wechseln dann automatisch.

13.11 Einsatz im modernen Internet

Ein großer Teil des Webverkehrs läuft heute über QUIC und HTTP/3, besonders bei großen Diensten, die ihre Infrastruktur darauf umgestellt haben. Viele Browser und Server unterstützen es, und die Anwendung wählt je nach Verfügbarkeit zwischen beiden Wegen. Für den Alltag bedeutet das: Wer im Browser eine Webseite öffnet, nutzt nicht zwingend TCP, sondern oft QUIC, ohne es zu bemerken.

> **Merke**
>
> QUIC ist ein eigenständiges, verschlüsseltes und zuverlässiges Transportprotokoll, das auf UDP aufsetzt. Es vereinigt Verbindungsaufbau und Verschlüsselung, trennt Streams voneinander und überlebt einen Adresswechsel.

Kontrollfragen

1. Auf welchem Protokoll setzt QUIC auf, und welchen Port nutzt es meist?
2. Warum fällt beim QUIC-Verbindungsaufbau eine Runde weg, verglichen mit TCP und TLS 1.2?
3. Welchen Vorteil hat das Multiplexing mit Streams gegenüber einer einzigen TCP-Verbindung?
4. Warum ist QUIC nicht einfach ein anderer Name für UDP?

Musterlösungen: 1. QUIC setzt auf UDP auf und nutzt meist Port 443. 2. TLS 1.3 ist in QUIC eingebaut, daher laufen Transportaufbau und Verschlüsselung gemeinsam in einer Runde. Bei TCP kommen TCP-Handshake und TLS-Handshake nacheinander. 3. Verliert ein Paket, bremst das nur den betroffenen Stream, die anderen laufen weiter. Bei TCP hält ein Verlust alle Daten der Verbindung zurück. 4. UDP bietet nur Ports und Prüfsumme. QUIC ergänzt Zuverlässigkeit, Reihenfolge, Überlastkontrolle, Verschlüsselung, Streams und Verbindungsmigration.

KAPITEL 14

Praktische Netzwerkdiagnose mit Wireshark

Bisher hast du Verbindungen über Zustände, Befehle und Beschreibungen kennengelernt. Wireshark macht sie sichtbar. Das Programm zeichnet den Netzverkehr auf dem eigenen Rechner auf und zeigt jedes Paket mit allen Feldern, die wir in den letzten Kapiteln besprochen haben. Ein Handshake, eine Wiederholung oder ein RST wird dort nicht mehr als Theorie sichtbar, sondern als konkrete Zeile. Dieses Kapitel führt dich schrittweise an die Paketdiagnose heran.

14.1 Wireshark installieren

Wireshark ist ein kostenloses Analysewerkzeug, das für Windows, Linux und macOS verfügbar ist. Lade es von der offiziellen Webseite herunter, nicht von fremden Quellen, und installiere es mit den Standardeinstellungen. Unter Windows kann die Installation den Paketfilter Npcap mitinstallieren, der für die Aufzeichnung nötig ist. Unter Linux installierst du das Paket über die Paketverwaltung deiner Distribution. Für die Aufzeichnung benötigt man unter Linux meist Administratorrechte oder die Gruppe, die den Zugriff erlaubt.

14.2 Netzwerkadapter auswählen

Beim Start zeigt Wireshark eine Liste der Netzwerkschnittstellen. Wähle die Schnittstelle, über die dein Rechner gerade kommuniziert. Bei einem Laptop ist das meist WLAN, bei einem Desktop-Rechner die Netzwerkkarte. Die Schnittstelle erkennst du an dem Verkehr, der nach Start der Aufzeichnung auftaucht. Auf einer falschen Schnittstelle bleibt die Liste leer.

14.3 Paketaufzeichnung starten

Mit einem Klick auf die Schnittstelle beginnt die Aufzeichnung. Die Pakete erscheinen in einer Liste, jede Zeile zeigt Nummer, Zeit, Quelle, Ziel, Protokoll und eine kurze Beschreibung. Die Aufzeichnung läuft, bis du sie stoppst. Achte darauf, nur den Verkehr deines eigenen Rechners oder ausdrücklich freigegebener Testsysteme aufzuzeichnen. Fremder Verkehr darf nicht ohne Erlaubnis mitgeschnitten werden.

Die folgende Abbildung zeigt schematisch, wie eine solche Paketliste aussieht. Es handelt sich um eine Darstellung, keinen echten Bildschirmabzug.

Nr.	Zeit	Quelle	Ziel	Protokoll	Information
1	0,000	192.168.1.10	198.51.100.20	TCP	51000 zu 443 [SYN]
2	0,031	198.51.100.20	192.168.1.10	TCP	443 zu 51000 [SYN, ACK]
3	0,031	192.168.1.10	198.51.100.20	TCP	51000 zu 443 [ACK]
4	0,032	192.168.1.10	198.51.100.20	TLSv1.3	Client Hello
5	0,044	192.168.1.10	192.0.2.53	DNS	Standard query A www.example.com

Schematische Darstellung, Zeiten und Adressen sind Beispiele

Abbildung 14.1. Schematische Darstellung einer Paketliste in Wireshark mit Handshake und einer DNS-Anfrage. Die Adressen stammen aus den Dokumentationsbereichen.

14.4 TCP-Verkehr erkennen

Im Feld Protokoll zeigt Wireshark TCP für Segmente der Transportschicht und die höheren Protokolle, die darauf aufsetzen, etwa TLS oder HTTP. Eine Zeile mit der Info Flags [SYN] zeigt den Anfang einer Verbindung. Wenn du einen Eintrag markierst, zeigt der untere Bereich alle Felder des Pakets, einschließlich Sequenznummer, Bestätigung, Flags und Optionen, wie in Kapitel 4 beschrieben.

14.5 UDP-Verkehr erkennen

UDP-Pakete erscheinen mit dem Protokoll UDP, oder mit dem darüberliegenden Dienst, etwa DNS, DHCP oder QUIC. Im Detailbereich siehst du die vier Felder des UDP-Headers. Der Unterschied zu TCP fällt auf: Es gibt keine Sequenznummern, keine Flags und keinen

Handshake.

14.6 TCP Three-Way Handshake untersuchen

Der Handshake ist die beste erste Übung. Starte die Aufzeichnung, öffne im Browser eine einfache Webseite, die du kennst, und stoppe danach. Filtere mit dem Anzeigefilter tcp.flags.syn == 1. Es erscheinen das SYN des Clients und das SYN-ACK des Servers. Das dritte Segment mit dem ACK erscheint, wenn du den Filter auf tcp und den passenden Port erweiterst. Vergleiche Sequenz- und Bestätigungsnummern mit dem Beispiel aus Kapitel 5.

14.7 TCP-Verbindungsabbau beobachten

Der Abbau zeigt FIN-Flags. Filtere mit tcp.flags.fin == 1 und prüfe, ob die Richtungen getrennt beendet werden. Bei einem sauberen Abbau siehst du vier Segmente. Ein RST zeigt sich mit tcp.flags.reset == 1 und bricht die Verbindung sofort ab.

14.8 Ports analysieren

Mit dem Filter tcp.port == 443 zeigst du nur Verkehr, der Port 443 als Quelle oder Ziel nutzt. Mit udp.port == 53 zeigst du DNS-Verkehr über UDP. Die Ports helfen, Dienste zu identifizieren, auch wenn die Anwendung selbst nicht bekannt ist.

14.9 TCP-Retransmissions erkennen

Wireshark markiert Wiederholungen automatisch, wenn es sie anhand der Sequenznummern erkennt. Der Filter tcp.analysis.retransmission zeigt nur diese Pakete. Viele Wiederholungen in kurzer Zeit deuten auf Verlust oder Überlast hin, wie wir in Kapitel 6 besprochen haben.

14.10 Duplicate ACKs erkennen

Duplicate ACKs erkennt Wireshark ebenfalls. Der Filter tcp.analysis.duplicate_ack zeigt Bestätigungen, die dieselbe Nummer wiederholen. Drei solche Bestätigungen gefolgt

von einer Wiederholung sind das typische Muster eines Fast Retransmit.

14.11 DNS-Kommunikation untersuchen

Eine DNS-Anfrage hat genau zwei Pakete, Anfrage und Antwort, und läuft über UDP auf Port 53. Filtere mit dns, um nur diese Pakete zu sehen. In der Antwort steht die aufgelöste Adresse. Vergleiche die Zeit zwischen beiden Paketen, sie entspricht ungefähr der Laufzeit zum DNS-Server.

14.12 Filter anwenden

Wireshark kennt zwei Arten von Filtern, die man nicht verwechseln sollte. Ein Anzeigefilter steuert, welche aufgezeichneten Pakete sichtbar sind, und verändert die Aufzeichnung nicht. Ein Aufzeichnungsfilter entscheidet vor der Aufzeichnung, welche Pakete gespeichert werden. Die folgende Tabelle fasst die wichtigsten Anzeigefilter zusammen.

Filter	Zeigt
tcp	alle TCP-Segmente
udp	alle UDP-Datagramme
tcp.port == 443	TCP-Verkehr mit Port 443
udp.port == 53	UDP-Verkehr mit Port 53
tcp.flags.syn == 1	Verbindungsaufbau, SYN
tcp.flags.reset == 1	Verbindungsabbruch mit RST
tcp.analysis.retransmission	Wiederholungen von Segmenten
tcp.analysis.duplicate_ack	Duplicate ACKs
dns	DNS-Nachrichten

Tabelle 14.1. Wichtige Anzeigefilter in Wireshark mit ihrer Wirkung.

Merke

Wireshark zeigt Pakete mit allen Feldern, die in diesem Band erklärt wurden. Ein Anzeigefilter wählt aus, was du sehen willst, ohne die Aufzeichnung zu verändern. Aufgezeichnet wird nur, was dir gehört oder wofür du die Erlaubnis hast.

Kontrollfragen

1. Welcher Anzeigefilter zeigt nur die SYN-Pakete einer Verbindung?
2. Welcher Filter zeigt die DNS-Anfrage eines Rechners?
3. Warum ist ein Anzeigefilter etwas anderes als ein Aufzeichnungsfilter?
4. Du siehst viele Retransmissions im Verkehr zu einem Server. Was kann die Ursache sein?

Musterlösungen: 1. tcp.flags.syn == 1. 2. dns oder udp.port == 53. 3. Der Anzeigefilter verbirgt Pakete nur in der Ansicht, die Aufzeichnung bleibt vollständig. Der Aufzeichnungsfilter entscheidet schon beim Mitschneiden, was gespeichert wird, und verwirft den Rest. 4. Mögliche Ursachen sind Paketverlust auf dem Weg, eine überlastete Leitung oder eine Störung am Server. Die Zeitpunkte der Wiederholungen helfen, die Ursache einzugrenzen.

KAPITEL 15

Praktische Laborübungen

Die Übungen in diesem Kapitel finden auf deinem eigenen Rechner oder in einer virtuellen Maschine statt. Alle Verbindungen laufen über localhost oder über ein internes Testnetz. Du sprichst also nur mit Diensten, die du selbst startest. Das ist wichtig, weil einige der Werkzeuge Verkehr mitschneiden oder Ports öffnen. Wer fremde Systeme untersucht, braucht eine ausdrückliche Erlaubnis.

Jede Übung ist gleich aufgebaut: Lernziel, Voraussetzungen, Schritte, Befehle, eine Beispielausgabe und die Erklärung der Ergebnisse. Am Ende steht eine Musterlösung. Die Beispielausgaben sind gekürzt und zeigen die Struktur. Deine eigenen Werte werden abweichen.

15.1 Vorbereitung

Für die Linux-Übungen brauchst du ein Terminal, für Wireshark eine Installation nach Kapitel 14. Die Windows-Befehle laufen in der Eingabeaufforderung oder in PowerShell. Einige Befehle benötigen Administratorrechte. Starte deine Testdienste in einem eigenen Terminal, damit du sie nicht versehentlich beendest.

15.2 Windows

Übung 1: Aktive TCP-Verbindungen untersuchen

Lernziel: Du erkennst, welche Verbindungen dein Rechner gerade offen hält, und ordnest ihnen Zustände zu.

Voraussetzungen: Ein Browser mit einer geöffneten Webseite.

Schritte: Öffne die Eingabeaufforderung und führe den Befehl aus. Suche eine Verbindung im Zustand ESTABLISHED und notiere lokale Adresse, Zieladresse und die Prozess-ID.

Befehl: `netstat -ano`

Beispielausgabe (gekürzt)

```
TCP    192.168.1.10:51000    198.51.100.20:443    ESTABLISHED    4812
```

Erklärung: Die Verbindung geht vom Client-Port 51000 zum Port 443 des Servers. Der Zustand ESTABLISHED zeigt, dass der Handshake abgeschlossen ist. Die Prozess-ID 4812 führt zum Programm, das die Verbindung hält. Mit dem Task-Manager oder tasklist findest du den Namen.

Musterlösung: Eine Zeile mit ESTABLISHED, dem Port 443 als Ziel und einer Prozess-ID, die du dem Browser zuordnen kannst.

Übung 2: Offene TCP-Ports anzeigen

Lernziel: Du erkennst Dienste, die auf Verbindungen warten.

Voraussetzungen: Keine.

Schritte: Führe in PowerShell als Administrator den Befehl aus und filtere auf den Zustand Listen.

Befehl: `Get-NetTCPConnection -State Listen`

Beispielausgabe (gekürzt)

```
LocalAddress   LocalPort   State
0.0.0.0        445         Listen
0.0.0.0        3389        Listen
```

Erklärung: Die Adresse 0.0.0.0 bedeutet, dass der Dienst auf allen Schnittstellen lauscht. Port 445 gehört zu SMB, Port 3389 zu RDP. Beide sollten nur dann von außen erreichbar sein, wenn es wirklich nötig ist.

Musterlösung: Du listest alle lauschenden Ports auf und ordnest jedem Port einen Dienst zu, zum Beispiel 445 SMB und 3389 RDP. Bei unbekannten Ports prüfst du den zugehörigen Prozess.

Übung 3: UDP-Ports anzeigen

Lernziel: Du erkennst UDP-Endpunkte, die auf Datagramme warten.

Voraussetzungen: Keine.

Schritte: Führe in PowerShell den Befehl aus und lies die lokalen Endpunkte.

Befehl: `Get-NetUDPEndpoint`

Beispielausgabe (gekürzt)

```
LocalAddress   LocalPort
0.0.0.0        68
0.0.0.0        5353
```

Erklärung: Port 68 ist der DHCP-Client, der auf Antworten wartet. Port 5353 gehört zu mDNS, einem Dienst für lokale Namen. UDP hat keinen Zustand wie LISTEN, deshalb zeigt der Befehl nur die Endpunkte.

Musterlösung: Du erkennst DHCP auf Port 68 und ordnest den zweiten Port einem Dienst zu. Ein Unterschied zu TCP ist, dass UDP keine Verbindung anzeigt, nur offene Endpunkte.

Übung 4: Einen TCP-Port testen

Lernziel: Du prüfst, ob ein bestimmter TCP-Port erreichbar ist.

Voraussetzungen: Ein lokaler Dienst auf Port 8000, den du in Übung 5 startest, oder ein eigener Webserver.

Schritte: Führe den Test gegen localhost aus und danach gegen einen Port, auf dem nichts läuft, zum Beispiel 8999.

Befehle: `Test-NetConnection -ComputerName 127.0.0.1 -Port 8000` und `Test-NetConnection -ComputerName 127.0.0.1 -Port 8999`

Beispielausgabe (gekürzt)

```
TcpTestSucceeded : True
TcpTestSucceeded : False
```

Erklärung: Der erste Test gelingt, weil ein Dienst lauscht. Der zweite schlägt fehl, weil auf Port 8999 nichts läuft. Ein schnelles Nein deutet auf einen geschlossenen Port hin, eine lange Wartezeit auf eine blockierte Verbindung, wie wir in Kapitel 5 gesehen haben.

Musterlösung: Der Test auf Port 8000 meldet True, der auf 8999 False. Dein Ergebnis entspricht dieser Logik.

15.3 Linux

Übung 5: Eine einfache TCP-Verbindung aufbauen

Lernziel: Du baust eine TCP-Verbindung zwischen zwei Programmen auf und siehst den Datenaustausch.

Voraussetzungen: Das Paket netcat, auf vielen Systemen vorinstalliert.

Schritte: Starte in einem Terminal einen Listener auf Port 9000. Verbinde dich im zweiten Terminal und schreibe eine Zeile. Die Zeile erscheint im ersten Terminal.

Befehle: `nc -l 9000` im ersten Terminal, `nc 127.0.0.1 9000` im zweiten Terminal.

Erklärung: Der Listener öffnet einen Socket an Port 9000. Der Client baut mit dem Handshake eine Verbindung auf. Was du tippst, geht als TCP-Daten zum Listener. Mit `ss -tan` siehst du während der Verbindung den Zustand ESTAB.

Musterlösung: Der Text erscheint im Listener, und `ss -tan` zeigt eine Verbindung mit Port 9000 im Zustand ESTAB.

Übung 6: TCP-Ports und Zustände anzeigen

Lernziel: Du unterscheidest lauschende Ports von bestehenden Verbindungen.

Voraussetzungen: Übung 5 läuft.

Schritte: Führe `ss -tulpen` aus, um lauschende Ports mit Prozessen zu sehen. Führe danach `ss -tan` aus, um alle TCP-Verbindungen mit ihrem Zustand zu sehen.

Befehle: `ss -tulpen` und `ss -tan`

Beispielausgabe (gekürzt)

```
tcp  LISTEN  0.0.0.0:9000  users:(("nc",pid=5123))
tcp  ESTAB   127.0.0.1:9000  127.0.0.1:52344
```

Erklärung: Der Listener zeigt LISTEN auf Port 9000 und den Prozess nc. Die Verbindung zeigt ESTAB mit dem Client-Port 52344. Beide Zeilen gehören zur gleichen Anwendung, aber nur die zweite ist eine echte Verbindung.

Musterlösung: Du findest den LISTEN-Eintrag und den ESTAB-Eintrag und erklärst den Unterschied in einem Satz.

Übung 7: UDP-Kommunikation testen

Lernziel: Du sendest Datagramme ohne Verbindung und beobachtest, dass keine Bestätigung kommt.

Voraussetzungen: netcat mit UDP-Unterstützung.

Schritte: Starte einen UDP-Listener auf Port 9001. Sende im zweiten Terminal eine Zeile an denselben Port. Stoppe danach den Listener und sende eine weitere Zeile.

Befehle: `nc -u -l 9001` und `nc -u 127.0.0.1 9001`

Erklärung: Solange der Listener läuft, kommt die Zeile an. Nach dem Beenden geht die Zeile verloren, und der Sender erfährt davon nichts. Es gibt keinen Handshake und keine Bestätigung. Genau das unterscheidet UDP von TCP, wie in Kapitel 10 beschrieben.

Musterlösung: Die erste Zeile erscheint, die zweite nicht. Der Sender bekommt in beiden Fällen keine Rückmeldung.

Übung 8: UDP-Datagramme untersuchen

Lernziel: Du siehst die Felder eines UDP-Datagramms im Mitschnitt.

Voraussetzungen: tcpdump, Administratorrechte, Übung 7.

Schritte: Starte tcpdump auf dem Loopback-Interface für Port 9001. Sende im zweiten Terminal eine Zeile.

Befehl: `sudo tcpdump -i lo -n udp port 9001`

Beispielausgabe (gekürzt)

```
IP 127.0.0.1.52001 > 127.0.0.1.9001: UDP, length 6
```

Erklärung: Das Datagramm kommt von Port 52001 an Port 9001. Die Länge 6 entspricht den Nutzdaten, der UDP-Header von acht Byte kommt dazu. Es gibt keine Sequenznummern und keine Flags.

Musterlösung: Du erkennst Quell- und Zielport, die Länge und das Fehlen aller TCP-Angaben.

Übung 9: Verbindungsaufbau mit Wireshark untersuchen

Lernziel: Du siehst den Three-Way Handshake als Paketfolge.

Voraussetzungen: Wireshark, Übung 5.

Schritte: Starte in Wireshark eine Aufzeichnung auf der Loopback-Schnittstelle. Baue die Verbindung aus Übung 5 neu auf. Setze den Anzeigefilter auf tcp.port == 9000.

Befehle: Anzeigefilter `tcp.port == 9000`

Erklärung: Du siehst drei Segmente: SYN vom Client, SYN-ACK vom Listener und ACK vom Client. Vergleiche Sequenz- und Bestätigungsnummern mit Kapitel 5. Die Bestätigung im zweiten Segment ist die Startnummer des Clients plus eins.

Musterlösung: Drei Segmente in der richtigen Reihenfolge, die Bestätigungsnummern stimmen mit der Logik des Handshakes überein.

Übung 10: Paketverlust und Retransmission erkennen

Lernziel: Du erzeugst einen Paketverlust in deinem eigenen Testnetz und erkennst die Wiederholung.

Voraussetzungen: Ein Linux-System, das du selbst verwaltest, mit Administratorrechten. Nur in einer eigenen virtuellen Maschine, niemals auf fremden Systemen.

Schritte: Simuliere auf der Schnittstelle eth0 einen Verlust von fünf Prozent. Übertrage danach eine größere Datei an einen lokalen Dienst und beobachte in Wireshark die Retransmissions. Entferne die Simulation am Ende.

Befehle: `sudo tc qdisc add dev eth0 root netem loss 5%`, danach `sudo tc qdisc del dev eth0 root`

Erklärung: Die Verlustsimulation verwirft zufällig Pakete. Wireshark zeigt Retransmissions und Duplicate ACKs, wie in Kapitel 6 beschrieben. Die Übertragung wird langsamer, weil das Congestion Window nach jedem Verlust sinkt.

Musterlösung: Im Filter tcp.analysis.retransmission erscheinen Wiederholungen. Nach dem Entfernen der Simulation verschwinden sie, und die Übertragung erreicht wieder die normale Geschwindigkeit.

Übung 11: Client-Server-Kommunikation nachvollziehen

Lernziel: Du verbindest Client und Server und verfolgst den gesamten Ablauf vom Verbindungsaufbau bis zum Abbau.

Voraussetzungen: Python 3 auf dem System.

Schritte: Starte einen einfachen Webserver im Verzeichnis deiner Wahl. Rufe ihn mit curl ab. Beobachte mit ss und tcpdump den Verlauf.

Befehle: `python3 -m http.server 8000` im ersten Terminal, `curl http://127.0.0.1:8000/` im zweiten.

Erklärung: Der Client öffnet eine Verbindung, sendet die HTTP-Anfrage und erhält die Antwort. Danach schließt die Verbindung geordnet. Im Zustand TIME-WAIT auf der Seite,

die zuerst schließt, siehst du den Nachhall des Abbaus, wie in Kapitel 9 erklärt.

Musterlösung: Du verfolgst Handshake, Anfrage, Antwort und Abbau in der Mitschnitt-Ausgabe und benennst die Phasen.

Übung 12: Abschlussübung zur Netzanalyse

Lernziel: Du wendest die Befunde aus den Übungen an, um einen Verbindungsproblem einzugrenzen.

Voraussetzungen: Die Übungen 1 bis 11.

Schritte: Starte einen Dienst auf Port 8000, stoppe ihn und versuche danach den Zugriff. Notiere, welche Fehlermeldung du erhältst und in welchem Zustand die Verbindung endet. Vergleiche das Ergebnis mit dem Test aus Übung 4.

Erklärung: Ein sofortiger Fehler wie Verbindung verweigert deutet auf einen geschlossenen Port und ein RST hin. Eine lange Wartezeit deutet auf eine blockierte Verbindung.

Musterlösung: Du beschreibst den Unterschied zwischen beiden Fällen und ordnest die Beobachtung der Ursache zu.

> **Merke**
>
> Die Übungen zeigen, dass Befehle und Mitschnitte dieselbe Geschichte erzählen. Netzstatus, Ports, Zustände und Pakete ergänzen einander. Wer alle drei Blickwinkel nutzt, findet Fehler schneller.

KAPITEL 16

Zusammenfassung und Prüfungsvorbereitung

Dieser Band hat die Transportschicht vollständig durchlaufen. Du kennst jetzt die Aufgaben von TCP und UDP, die Bedeutung der Ports, den Aufbau der beiden Header, den Ablauf des Verbindungsaufbaus und -abbaus, die Mechanismen zur Zuverlässigkeit, Fluss- und Überlastkontrolle sowie den modernen Ansatz QUIC. Dieses Kapitel fasst das Wichtigste zusammen und gibt dir Gelegenheit, dein Wissen zu prüfen. Versuche die Aufgaben zuerst ohne Blick in die Lösungen.

16.1 Wichtigste TCP-Begriffe

TCP ist verbindungsorientiert und zuverlässig. Es nummeriert jedes Byte, bestätigt den Empfang kumulativ und wiederholt verlorene Segmente. Drei Duplicate ACKs lösen einen Fast Retransmit aus, ein Timeout ist die letzte Rückfallebene. Die Flusskontrolle begrenzt den Sender durch das Receive Window, die Überlastkontrolle durch das Congestion Window. Der Sender darf höchstens so viel senden, wie beide Grenzen zusammen erlauben.

16.2 Wichtigste UDP-Begriffe

UDP ist verbindungslos und unzuverlässig. Es bietet Ports und eine optionale Prüfsumme und sonst nichts. Ein Datagramm kommt entweder vollständig an oder gar nicht, eine Wiederholung muss die Anwendung selbst übernehmen. UDP ist schnell und passt zu Anwendungen, bei denen Aktualität wichtiger ist als Vollständigkeit.

16.3 TCP-Header und UDP-Header

Der TCP-Header ist mindestens 20 Byte groß und enthält Ports, Sequenz- und Bestätigungsnummer, Data Offset, Flags, Window Size, Prüfsumme und Urgent Pointer, dazu Optionen. Der UDP-Header ist acht Byte groß und enthält nur Ports, Länge und Prüfsumme. Die Prüfsumme ist bei UDP über IPv4 optional und über IPv6 Pflicht.

16.4 TCP-Handshake

Der Three-Way Handshake besteht aus SYN, SYN-ACK und ACK. Er synchronisiert die Startnummern beider Seiten, bestätigt die Verbindung in beide Richtungen und handelt Optionen wie MSS, Window Scaling, SACK Permitted und Timestamps aus. Verbindungen, die keine Antwort erhalten, werden mit wachsendem Abstand wiederholt. Ein RST als Antwort zeigt einen geschlossenen Port.

16.5 Abbau und Zustände

Der geordnete Abbau umfasst vier Segmente und läuft in beide Richtungen getrennt. TIME-WAIT schützt die Verbindung vor verspäteten Segmenten, CLOSE-WAIT zeigt eine Anwendung, die nicht schließt. RST bricht die Verbindung sofort ab und verwirft ausstehende Daten.

16.6 QUIC

QUIC setzt auf UDP auf und integriert Verschlüsselung und Verbindungsaufbau. Es trennt Streams voneinander, vermeidet dadurch Head-of-Line Blocking über die gesamte Verbindung und überlebt einen Adresswechsel über die Connection ID.

16.7 Wichtigste Formeln und Werte

Die folgenden Angaben reichen für die meisten Aufgaben aus.

- Portbereiche: 0 bis 1023 Well-Known, 1024 bis 49151 Registered, 49152 bis 65535 Dynamic.
- Mindestlänge TCP-Header: 20 Byte. Data Offset 5 entspricht 20 Byte ohne Optionen.
- UDP-Header: 8 Byte. Maximale UDP-Nutzdaten über IPv4: 65507 Byte.
- Typische MSS über Ethernet: 1460 Byte.
- Ein SYN und ein FIN verbrauchen je eine Sequenznummer.
- Bestätigungsnummer = nächstes erwartetes Byte.

16.8 Multiple-Choice-Fragen

Bei jeder Frage ist genau eine Antwort richtig.

1. Welcher Port ist der Standardport für HTTPS?
   a) 80 b) 443 c) 53 d) 3389
2. Welches Flag bricht eine TCP-Verbindung sofort ab?
   a) FIN b) PSH c) RST d) URG
3. Wie viele Segmente umfasst der normale Verbindungsaufbau bei TCP?
   a) zwei b) drei c) vier d) fünf
4. Welche Länge hat der UDP-Header?
   a) 8 Byte b) 16 Byte c) 20 Byte d) 60 Byte
5. Was löst einen Fast Retransmit aus?
   a) ein Timeout b) drei Duplicate ACKs c) ein RST d) ein Zero Window
6. Welches Protokoll setzt QUIC auf?
   a) TCP b) IP c) UDP d) ICMP
7. Welcher Zustand zeigt, dass die Gegenseite ihr FIN gesendet hat, die lokale Anwendung aber noch nicht geschlossen hat?
   a) TIME-WAIT b) CLOSE-WAIT c) LISTEN d) SYN-SENT
8. Welcher Filter zeigt in Wireshark nur SYN-Pakete?
   a) tcp.flags.reset == 1 b) tcp.flags.syn == 1 c) udp d) dns

16.9 Offene Fragen

1. Warum nummeriert TCP jedes Byte und nicht jedes Segment?
2. Erkläre den Unterschied zwischen Flusskontrolle und Überlastkontrolle an einem Beispiel.
3. Warum bleibt die aktive Seite nach dem Abbau im Zustand TIME-WAIT?
4. Nenne zwei Gründe, warum UDP für Sprachübertragungen geeignet ist, und einen Grund, warum TCP für Dateien geeignet ist.
5. Warum ist QUIC nicht einfach ein anderer Name für UDP?
6. Wie unterscheidet sich eine Verbindung, die durch Timeout scheitert, von einer, die durch RST abgelehnt wird?

16.10 Praxisaufgaben

1. Führe auf deinem Rechner netstat oder ss aus und ordne drei offene Ports den Diensten zu. Begründe jede Zuordnung mit der Portnummer.
2. Starte einen einfachen Dienst auf Port 8000 und teste ihn mit einem Verbindungstest. Notiere, was passiert, wenn du den Dienst stoppst.
3. Zeichne den Three-Way Handshake und den Abbau für eine eigene Verbindung auf Papier und vergleiche die Sequenznummern mit einem Mitschnitt.
4. Erkläre in eigenen Worten, welches Protokoll du für einen Videostream, für eine Dateiübertragung und für eine DNS-Anfrage wählen würdest und warum.

16.11 Vollständige Lösungen

Multiple-Choice

Die richtigen Antworten lauten: 1 b (443), 2 c (RST), 3 b (drei Segmente), 4 a (8 Byte), 5 b (drei Duplicate ACKs), 6 c (UDP), 7 b (CLOSE-WAIT), 8 b (tcp.flags.syn == 1).

Offene Fragen

TCP nummeriert Bytes, weil Segmente unterschiedlich groß sein können und sich beim Neusenden zu Teilen zerlegen oder zusammenfassen lassen. Mit einer Nummer pro Byte kann der Empfänger jede Lücke exakt bestimmen und die Daten sortieren.

Flusskontrolle schützt den Empfänger. Ein Sender mit schnellem Netz, aber einem langsamen Empfänger wird durch das Receive Window gebremst. Überlastkontrolle schützt das Netz. Ein Sender, dessen Pakete an einem überfüllten Router verloren gehen, wird durch das Congestion Window gebremst, auch wenn der Empfänger viel Platz hat.

TIME-WAIT verhindert, dass verspätete Segmente der alten Verbindung in eine neue mit denselben Adressen und Ports gelangen. Die Wartezeit ist ein Schutz gegen Verwechslung.

Für Sprache sind UDP-Datagramme geeignet, weil Verzögerung schädlicher ist als ein verlorenes Stück, und ein verspätetes Stück wertlos wäre. Für Dateien ist TCP geeignet, weil jedes Byte fehlen würde und die Datei unbrauchbar wäre.

QUIC ergänzt UDP um Zuverlässigkeit, Reihenfolge, Überlastkontrolle, Verschlüsselung, Streams und Verbindungsmigration. UDP bietet nur Ports und Prüfsumme.

Eine Verbindung, die durch Timeout scheitert, bekommt keine Antwort. Eine durch RST

abgelehnte Verbindung bekommt eine sofortige Antwort, die den Abbruch mitteilt. Der Unterschied ist ein wichtiger Hinweis bei der Fehlersuche.

Praxisaufgaben

Die Ergebnisse hängen von deinem System ab. Achte darauf, dass jede Zuordnung über die Portnummer begründet ist, etwa Port 443 für HTTPS. Beim Dienst auf Port 8000 zeigt der Test nach dem Start True, nach dem Stoppen False. Der Handshake und Abbau sind mit Papier und Mitschnitt nachvollziehbar, die Sequenznummern steigen um die Anzahl der Nutzdaten. Für den Videostream bietet sich UDP oder QUIC an, für die Dateiübertragung TCP, für DNS UDP mit TCP als Rückfall bei großen Antworten.

Anhang

Der Anhang bündelt Nachschlagewerte, die du bei Übungen, in der Praxis und bei der Prüfungsvorbereitung brauchst. Die Tabellen lassen sich schnell durchsehen, deshalb stehen die Angaben kurz und ohne Erklärung. Die ausführlichen Erklärungen findest du in den jeweiligen Kapiteln.

Spickblatt Transportschicht

TCP ist verbindungsorientiert und zuverlässig, UDP ist verbindungslos und schlank. Ports kennzeichnen Anwendungen, eine Verbindung ist durch vier Angaben eindeutig, nämlich Quell- und Zieladresse sowie Quell- und Zielport. Der Handshake besteht aus SYN, SYN-ACK und ACK, der Abbau aus vier Segmenten. Drei Duplicate ACKs lösen einen Fast Retransmit aus. Die Sendemenge ist durch das kleinere von Receive Window und Congestion Window begrenzt.

Portbereiche

Bereich	Zweck
0 bis 1023	Well-Known Ports, allgemein bekannte Dienste
1024 bis 49151	Registered Ports, von Anwendungen registriert
49152 bis 65535	Dynamic oder Private Ports, flüchtige Ports für Verbindungen

Wichtige Ports

Port	Protokoll	Dienst
20, 21	TCP	FTP, Daten und Steuerung
22	TCP	SSH
23	TCP	Telnet, unverschlüsselt
25	TCP	SMTP, E-Mail-Versand
53	UDP, TCP	DNS
67, 68	UDP	DHCP, Server und Client
80	TCP	HTTP
110	TCP	POP3
123	UDP	NTP
143	TCP	IMAP
443	TCP, UDP	HTTPS, bei QUIC über UDP
445	TCP	SMB, Dateifreigaben
993	TCP	IMAP über TLS
3389	TCP	RDP, Remotedesktop

TCP-Header im Überblick

Feld	Größe	Zweck
Source Port	16 Bit	Anwendung des Absenders
Destination Port	16 Bit	Anwendung des Empfängers
Sequence Number	32 Bit	Nummer des ersten Datenbytes
Acknowledgment Number	32 Bit	nächstes erwartetes Byte
Data Offset	4 Bit	Headerlänge in 32-Bit-Wörtern
Reserved	3 Bit	für künftige Nutzung
Flags	9 Bit	Steuerung, unter anderem SYN, ACK, FIN, RST
Window Size	16 Bit	Größe des Empfangsfensters

Feld	Größe	Zweck
Checksum	16 Bit	Prüfung von Header und Daten
Urgent Pointer	16 Bit	nur bei gesetztem URG
Options und Padding	bis 40 Byte	MSS, Window Scaling, SACK, Timestamps

UDP-Header im Überblick

Feld	Größe	Zweck
Source Port	16 Bit	optional, Antwortport
Destination Port	16 Bit	Dienst des Empfängers
Length	16 Bit	Länge von Header und Daten
Checksum	16 Bit	bei IPv4 optional, bei IPv6 Pflicht

Protokollnummern im IP-Header

Nummer	Protokoll
1	ICMP für IPv4
6	TCP
17	UDP
58	ICMPv6, im Next-Header-Feld von IPv6

TCP-Zustände

Zustand	Kurz erklärt
CLOSED	keine Verbindung
LISTEN	Server wartet auf SYN
SYN_SENT	SYN gesendet, wartet auf SYN-ACK
SYN_RECEIVED	SYN empfangen, SYN-ACK gesendet
ESTABLISHED	Verbindung steht
FIN_WAIT_1 und FIN_WAIT_2	aktive Seite im Abbau
CLOSE_WAIT	Gegenseite hat FIN gesendet, Anwendung schließt noch nicht
CLOSING	gleichzeitiges FIN beider Seiten
LAST_ACK	passive Seite wartet auf die letzte Bestätigung
TIME_WAIT	aktive Seite wartet zweimal die MSL

Wichtige Richtwerte

Wert	Angabe
MSS über Ethernet	1460 Byte
Maximale Nutzdaten UDP über IPv4	65507 Byte
Maximaler Window-Scale-Faktor	14
Initial Window nach RFC 6928	meist 10 Segmente
Mindest-RTO nach RFC 6298	1 Sekunde
Duplicate ACKs für Fast Retransmit	3
Standard-MSL nach RFC 793	2 Minuten, TIME_WAIT also 4 Minuten

Glossar

Begriff	Erklärung
Acknowledgment	Bestätigung des Empfangs, nennt das nächste erwartete Byte
Congestion Window	vom Sender geschätzte Menge unbestätigter Daten im Netz
Datagramm	Dateneinheit von UDP
Duplicate ACK	wiederholte Bestätigung, zeigt eine Lücke an
Fast Retransmit	sofortige Wiederholung nach drei Duplicate ACKs
Flusskontrolle	Schutz des Empfängers durch das Receive Window
Handshake	Verbindungsaufbau aus SYN, SYN-ACK und ACK
Head-of-Line Blocking	Verzögerung nachfolgender Daten durch ein verlorenes Paket
Initial Sequence Number	zufällige Startnummer einer Verbindung
MSS	größte Nutzdatenmenge eines TCP-Segments
Port	Nummer, die eine Anwendung auf einem Gerät kennzeichnet
QUIC	verschlüsseltes Transportprotokoll auf UDP
Retransmission	erneutes Senden eines als verloren geltenden Segments
RTO	Retransmission Timeout, Wartezeit vor der Wiederholung
RTT	Round Trip Time, Zeit bis zur Bestätigung
SACK	selektive Bestätigung bestimmter Datenbereiche
Segment	Dateneinheit von TCP
Sequenznummer	Nummer des ersten Datenbytes eines Segments
Sliding Window	gleitendes Fenster, das die erlaubte Sendemenge bestimmt
Socket	Schnittstelle, über die eine Anwendung das Netz nutzt
Timeout	Ablauf der Wartezeit ohne Bestätigung
Zero Window	Empfangsfenster von null, der Sender muss warten

Abkürzungsverzeichnis

Abkürzung	Bedeutung
ACK	Acknowledgment
AIMD	Additive Increase Multiplicative Decrease
BBR	Bottleneck Bandwidth and Round-trip propagation time

Abkürzung	Bedeutung
CUBIC	TCP-Überlastverfahren mit kubischer Fensterfunktion
ECN	Explicit Congestion Notification
FIN	Finish, Flag zum Ende einer Richtung
HTTP/3	HTTP über QUIC
IANA	Internet Assigned Numbers Authority
ISN	Initial Sequence Number
MSL	Maximum Segment Lifetime
MSS	Maximum Segment Size
NTP	Network Time Protocol
QUIC	Quick UDP Internet Connections
RFC	Request for Comments
RST	Reset, Flag zum sofortigen Abbruch
RTO	Retransmission Timeout
RTT	Round Trip Time
SACK	Selective Acknowledgment
SYN	Synchronize, Flag zum Verbindungsaufbau
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol

Wichtige Standards und RFCs

- RFC 768: User Datagram Protocol.
- RFC 793: Transmission Control Protocol, durch RFC 9293 ersetzt.
- RFC 2018: TCP Selective Acknowledgment Options.
- RFC 3168: The Addition of Explicit Congestion Notification to IP.
- RFC 5681: TCP Congestion Control.
- RFC 6298: Computing TCP’s Retransmission Timer.
- RFC 6528: Defending against Sequence Number Attacks.
- RFC 6928: Increasing TCP’s Initial Window.
- RFC 7323: TCP Extensions for High Performance, Window Scaling und Timestamps.
- RFC 8312: CUBIC for Fast Long-Distance Networks.
- RFC 9000: QUIC, Transport.
- RFC 9001: Using TLS to Secure QUIC.

- RFC 9002: QUIC Loss Detection and Congestion Control.
- RFC 9114: HTTP/3.
- RFC 9293: Transmission Control Protocol, Überarbeitung von RFC 793.

Weiterführende Lernressourcen

Die Portliste der IANA ist die verbindliche Quelle für Portzuordnungen. Die Dokumentation von Wireshark enthält ausführliche Beschreibungen aller Filter und Protokolle. Microsoft Learn beschreibt die Netzwerkbefehle unter Windows, die Handbuchseiten von Linux erklären ss, netcat, tcpdump und tc. Die RFCs selbst sind bei der IETF frei verfügbar und lohnen die Lektüre, besonders RFC 9293 für TCP und RFC 9000 für QUIC.

NETZWERK VERSTEHEN • BAND 3

TCP und UDP – Die Transportprotokolle

Wenn du eine Webseite lädst, ein Video streamst oder eine Datei verschickst, entscheidet die Transportschicht, ob Daten vollständig, in der richtigen Reihenfolge und zur richtigen Anwendung gelangen. Dieser Band erklärt die beiden wichtigsten Protokolle dafür: TCP und UDP.

Das Buch richtet sich an Einsteiger, Quereinsteiger, Umschüler, Auszubildende und Administratoren. Jeder Ablauf wird Schritt für Schritt erklärt und mit einer Grafik veranschaulicht, vom Dreiwege-Handshake bis zur Wiederholung verlorener Daten.

Du lernst, Ports und Sockets zu deuten, TCP-Header und UDP-Datagramme zu lesen, Flusskontrolle und Überlastkontrolle zu verstehen und Verbindungen mit Befehlen und Wireshark zu untersuchen. Die Laborübungen funktionieren auf dem eigenen Rechner oder in einer virtuellen Maschine.

Wer die Transportschicht versteht, erkennt Verbindungsprobleme schneller und ordnet Netzwerkverkehr richtig ein.

Band 3 der Reihe *Netzwerk verstehen* baut auf den Bänden 1 und 2 auf, die Netzwerktechnik und IP behandeln. Er schließt die Reihe mit der Transportschicht ab.

Eugen Becker